

- Quantifying and Mitigating SZZ-Induced Label Noise in Just-In-Time Software Defect Prediction under Verification-Latency-Aware Online Evaluation
 - Abstract
 - Abbreviations
 - A note on corrections
- Chapter 1 — Introduction
 - 1.1 Context
 - 1.2 The ground-truth problem
 - 1.3 The realism problem
 - 1.4 Problem statement
 - 1.5 Research gap
 - 1.6 Research questions
 - 1.7 Contributions
 - 1.8 Thesis organisation
- Chapter 2 — Background and Related Work
 - 2.1 Just-in-time software defect prediction
 - 2.2 SZZ and its variants
 - 2.3 Evaluation methodology in defect prediction
 - 2.4 Online learning for JIT-SDP
 - 2.5 The weak-baseline problem
 - 2.6 Metric selection
 - 2.7 Learning with noisy labels
 - 2.8 Positioning
 - 2.9 Citation verification status
- Chapter 3 — Research Design
 - 3.1 The four-phase pipeline
 - 3.2 Data
 - 3.3 SZZ variant implementations
 - 3.4 Metrics and statistical protocol
 - 3.5 Threats to validity, framed in advance
 - 3.6 Reproducibility
- Chapter 4 — Phase 1: Label Disagreement and Oracle Comparison
 - 4.1 Method
 - 4.2 Results
 - 4.3 The bias model
 - 4.4 Discussion — answering RQ1
- Chapter 5 — Phase 2: Downstream Impact Under Honest Evaluation
 - 5.1 Method
 - 5.2 Results

- 5.3 An honest negative: the batch-to-stream contrast could not be isolated
- 5.4 Robustness
- 5.5 Discussion — answering RQ2
- Chapter 6 — Phase 3: Diagnosing Learner Behaviour Under Noise
 - 6.1 Method
 - 6.2 Results
 - 6.3 Discussion — answering RQ3
- Chapter 7 — Phase 4: Noise-Aware ORB
 - 7.1 Pre-registration, and a registration that was rewritten
 - 7.2 Design
 - 7.3 Experimental setup
 - 7.4 Results
 - 7.5 Discussion — answering RQ4
- Chapter 8 — Discussion
 - 8.1 Synthesis across the research questions
 - 8.2 Implications
 - 8.3 Threats to validity
- Chapter 9 — Conclusion and Future Work
 - 9.1 What this thesis set out to do
 - 9.2 Contributions
 - 9.3 Limitations
 - 9.4 Future work
 - 9.5 Closing

Quantifying and Mitigating SZZ-Induced Label Noise in Just-In-Time Software Defect Prediction under Verification-Latency-Aware Online Evaluation

Puneet Deshwani — M.Tech Thesis

Abstract

Just-in-time software defect prediction (JIT-SDP) labels almost universally descend from the SZZ algorithm, which infers defect-introducing commits by blaming the lines a bug fix modifies. That inference fails whenever bug-fixing commits are tangled, and the field has no measurement of what the resulting noise costs a model under realistic deployment conditions — in part because models are conventionally evaluated against the same SZZ labels that trained them.

This thesis separates four quantities across 21 Apache Java projects and 27,319 commits, holding out an independently constructed reference label set so that a model trained on SZZ can be scored against something other than SZZ.

Label quality. Six SZZ variants are scored over an identical denominator. No variant exceeds **27.2% precision**. Noise is class-conditional rather than symmetric (false-alarm rates 6.7–26.3%, miss rates 35.9–73.3%). Variants agree with one another at κ up to **0.933** while agreeing with the reference at κ **0.158–0.202** — they share failure modes, so mutual agreement is not evidence of correctness.

Evaluation protocol. Random k -fold cross-validation inflates a high-capacity model by **+0.137 MCC** (21 of 21 projects); scoring against the training labels inflates it by a further **+0.230**. The two protocol effects together are roughly nine times the label-source effect, and both scale with model capacity. A result reading 0.41 MCC under the field’s standard configuration is **0.10** under a defensible one.

Mechanism. A controlled repair experiment shows that removing SZZ’s false positives recovers **+0.040 MCC** while restoring its false negatives is statistically *equivalent to no repair*. An error-matched control establishes this as an effect of error **volume**, not per-label severity. The mechanism is measured: the online learner’s oversampling rate is a near-perfect inverse of delivered defect-label supply ($\rho = -1.000$), so the machinery that corrects class imbalance is the machinery that amplifies wrong labels.

Mitigation. A pre-registered train-time false-positive filter passes its non-degradation gate but no confirmatory test survives correction. The gap between the oracle-assisted repair (+0.040) and its deployable approximation (+0.017) quantifies the price of not knowing which labels are wrong: a confidence signal cannot separate a wrong label from an unlearned pattern.

On this evidence, **improving the labels is worth more than improving the learner.**

Abbreviations

AUC	Area under the ROC curve
CI	Confidence interval
CL	Confident Learning
FN / FP / TN / TP	False negative / false positive / true negative / true positive
HL	Hodges–Lehmann estimator
JIT-SDP	Just-in-time software defect prediction
MCC	Matthews correlation coefficient
ND	Non-degradation (Phase 4 acceptance gate)
OOB	Oversampling Online Bagging
ORB	Oversampling Rate Boosting
SZZ	Śliwerski–Zimmermann–Zeller algorithm (variants: B-, AG-, MA-, R-, L-, RA-)
TOST	Two one-sided tests (equivalence testing)
ρ_0 / ρ_1	Class-conditional false-alarm rate / miss rate
λ	Poisson oversampling rate in online bagging
W	Verification-latency decision window (90 days)

A note on corrections

Several findings in this thesis contradict claims made earlier in the same project: the false-negative starvation hypothesis Phase 4 was originally designed around, a decomposition of the verification-latency effect, and a conclusion about whether latency masks label quality. In each case the superseded claim is stated together with the design error that produced it, rather than removed.

This is deliberate. The corrections are part of the evidence — they record which conclusions survived contact with a control and which did not, and in three of the four cases the control that overturned the claim had been built for a different purpose. A caveat is not a control, and most of the substantive revisions in this work exist because that distinction was eventually enforced.

Chapter 1 — Introduction

1.1 Context

Defects are expensive in proportion to how long they survive. A fault caught at review costs a developer’s attention; the same fault caught in production costs an incident, a rollback and the trust of whoever depended on the system. This asymmetry motivates *defect prediction*: statistical models that direct limited inspection effort toward the code most likely to be wrong.

Classical defect prediction operates at release granularity, predicting which files or modules will contain defects. Its practical difficulty is one of timing. A prediction about a file, delivered at release, arrives long after the change that introduced the problem, and often reaches a developer who did not write it.

Just-in-time software defect prediction (JIT-SDP) moves the unit of prediction from the file to the *commit*. Each change is scored as it arrives, so a risky change can be flagged while its author still has it in mind. Kamei et al. (2013) established the modern form of the task, defining a set of change-level metrics — size, diffusion, history, and developer experience — that remain the standard feature set, and reporting roughly 68% accuracy and 64% recall at predicting defect-inducing changes.

The setting is unusual in three ways that jointly shape everything in this thesis. Commits arrive as a **stream**, so any realistic evaluation is temporal. Defect-introducing commits are **rare** — 8.54% on the corpus studied here — so the task is severely imbalanced. And, critically, **the label for a commit is not available when the commit arrives**. It becomes available only when someone finds the defect, traces it to its origin, and fixes it.

1.2 The ground-truth problem

JIT-SDP requires a label for every commit: did this change introduce a defect? Almost no project records this directly. What projects record is which commits *fixed* defects.

The gap is bridged by the **SZZ algorithm** (Śliwerski, Zimmermann and Zeller), which works backwards: take a bug-fixing commit, identify the lines it modified, and use `git blame` to find which earlier commit last touched each of those lines. Those earlier commits are labelled defect-introducing.

The inference is only as good as its premise — that the lines a fix modifies are the lines that were wrong. **That premise fails routinely, because bug-fixing commits are tangled.** Developers reformat while fixing, rename while fixing, refactor while fixing, update comments and tests while fixing. Every such line is blamed by SZZ, and every commit

that last touched one is labelled as having introduced a defect it had nothing to do with.

The magnitude is now measured rather than suspected. Herbold et al. (2022), using four annotators per line, found that **between 17% and 32% of all changes within bug-fixing commits actually address the underlying problem** — rising to 66–87% when only production code files are considered. The remainder is tangled. Roughly 11% of lines were hard enough to classify that annotators actively disagreed.

A substantial body of work has proposed refinements — AG-SZZ, MA-SZZ, R-SZZ, L-SZZ, RA-SZZ — each adding filters intended to remove the over-collection. Whether those refinements improve the labels, and at what cost, is the subject of Chapter 4.

The consequence for the field is structural. Essentially every JIT-SDP dataset in use descends from SZZ. Models are trained on SZZ labels, evaluated against SZZ labels, and compared with one another on SZZ labels. If those labels are systematically wrong, the entire measurement apparatus is calibrated against a biased instrument — and, because the evaluation uses the same instrument as the training, the bias is invisible from inside.

1.3 The realism problem

A second problem is independent of labels and compounds them.

The standard evaluation protocol in the JIT-SDP literature is random k -fold cross-validation: shuffle the commits, hold out a fold, train on the rest. On temporally ordered data this permits a model to train on 2020 commits and be tested on 2015 ones. The resulting estimate answers a question nobody has: how well would this model do if it could see the future?

Verification latency is the subtler half. Even under a correct chronological split, the assumption that a commit’s label is available at training time is false. The label exists only once the defect is found and fixed, and on this corpus the median delay is 113 days, with **53% of defect labels arriving after a 90-day decision window**.

This is worse than a delay. A system with a 90-day window does not simply wait — it concludes that an unfixed commit is clean, trains on that conclusion, and is corrected months later. **A learner under verification latency is actively trained on labels it will subsequently discover were wrong**, and the wrong labels it receives are exactly the defect-introducing commits it most needs to learn from.

Cabral and Minku (2019) characterised this setting, showing that the class imbalance itself evolves over time under latency, and proposed **Oversampling Rate Boosting (ORB)** — an online ensemble that adjusts its resampling rate when predictions become biased toward one class. ORB is the online learner used throughout this thesis, and the mechanism it uses to handle imbalance turns out, in Chapter 6, to be the mechanism through which label noise does its damage.

1.4 Problem statement

Just-in-time defect prediction is evaluated almost exclusively against labels produced by SZZ, using protocols that ignore both commit order and verification latency. The field therefore has **no measurement of what SZZ-induced label noise costs a model under realistic deployment conditions**, and no way to distinguish a model’s ability to find defects from its ability to reproduce the heuristic that labelled them.

Four things must be separated to make progress: the noise in the labels; the inflation contributed by the evaluation protocol; the *mechanism* by which noise damages an online learner; and whether a learner can defend itself. This thesis addresses them in that order, as four phases.

1.5 Research gap

Three lines of work approach this territory without closing it.

SZZ evaluation research measures how accurately SZZ variants identify defect-introducing commits, including against developer-informed oracles. It establishes that the labels are noisy. It stops at the labels and does not measure the downstream cost.

Online JIT-SDP research addresses class imbalance evolution and verification latency, and Cabral and Minku’s ORB includes a safety mechanism against potentially noisy minority-class examples. This work treats noise as a nuisance to be dampened rather than as a measurable quantity with a known, asymmetric structure and a known origin.

Learning with noisy labels provides loss correction, sample selection and confidence-based methods. These are almost entirely batch methods developed for symmetric or uniform label noise, and do not transfer directly to a stream in which labels arrive late, arrive wrong, and are later corrected.

The gap is the intersection. No existing work measures the cost of *SZZ-origin* noise — asymmetric, variant-dependent, with known flip rates — under *latency-aware online* evaluation, and no work isolates which of SZZ’s two error types an online learner actually cannot survive. This thesis occupies that intersection, and its central methodological device is the one that makes the intersection measurable: **holding out an independently constructed reference label set so that a model trained on SZZ can be scored against something other than SZZ.**

1.6 Research questions

RQ1. How much do SZZ variants disagree with one another and with an independently constructed reference, and what is the direction of each variant’s labelling bias? (*Chapter 4*)

RQ2. How does the choice of SZZ label source shift measured JIT-SDP performance across naive, chronological, and prequential-with-latency evaluation regimes? (*Chapter 5*)

RQ3. Through what mechanism does label noise degrade an online learner — specifically, how does ORB’s oversampling response behave under symmetric versus SZZ-calibrated asymmetric noise as the dose increases? (*Chapter 6*)

RQ4. Can an online learner recover performance lost to SZZ-origin noise without access to clean labels, and without sacrificing performance when the labels are clean? (*Chapter 7*)

1.7 Contributions

A class-conditional characterisation of SZZ noise. Six variants scored over an identical 27,319-commit denominator: a precision ceiling of 27.2%, bifurcated bias (p_0 0.067–0.263, p_1 0.359–0.733), and inter-variant agreement of κ up to 0.933 against reference agreement of κ 0.158–0.202. **Variants share failure modes, so their mutual agreement is not evidence of correctness.** The flip-rate pairs are released as a calibration artefact.

A separation of protocol inflation from label effects. Random k -fold inflates a high-capacity model by +0.137 MCC (21 of 21 projects); self-scoring inflates it by a further +0.230. The two protocol effects together are roughly nine times the label-source effect, and the inflation scales with model capacity.

A controlled identification of which SZZ error costs an online learner, and why. Repairing false positives recovers +0.040 MCC; restoring false negatives is *equivalent to no repair*. An error-matched control establishes this as a volume effect rather than a per-label one, and the mechanism is measured: the learner’s oversampling rate is a near-perfect inverse of delivered defect-label supply.

A pre-registered negative result with a diagnosis. A train-time false-positive filter is adoptable but not demonstrably effective, and the gap between the oracle-assisted repair and its deployable approximation quantifies the price of not knowing which labels are wrong.

A reproducible pipeline with an integrity gate. The full study runs on a continuous-integration runner from two small committed files, behind an assertion that recomputes the training data’s confusion matrix and refuses to spend compute if it disagrees with the reported noise rates.

1.8 Thesis organisation

Chapter 2 surveys JIT-SDP, SZZ and its variants, evaluation methodology, online learning under latency, and learning with noisy labels, and positions this work against each.

Chapter 3 states the research design: the four-phase pipeline, the corpus and its reference labels, the SZZ toolchain, the metrics and statistical protocol, and the validity framework — stated in advance rather than assembled afterwards.

Chapters 4 to 7 report the four phases, one research question each.

Chapter 8 synthesises the four results into a single account and gives the full treatment of validity threats.

Chapter 9 concludes.

A note on how the results chapters are written. Several findings in this thesis contradict claims made earlier in the same project — the starvation hypothesis Phase 4 was designed around, a decomposition of the latency effect, a conclusion about whether latency masks label quality. In each case the superseded claim is **stated, together with the design error that produced it**, rather than removed. The corrections are part of the evidence: they show which conclusions survived contact with a control and which did not.

Chapter 2 — Background and Related Work

Citation status. §2.9 classifies every reference in this chapter as [V] — bibliographic details and the specific claim attributed to it were verified against the source or its publisher record during preparation — or [C] — the work is correctly identified but the attributed detail has **not** been re-verified and must be checked against the original before submission. No claim marked [C] should be quoted without that check.

2.1 Just-in-time software defect prediction

2.1.1 Task definition

JIT-SDP predicts, for each incoming commit, whether it introduces a defect. Formally, given a commit c with feature vector $x \in \mathbb{R}^d$ available at commit time, predict $y \in \{0, 1\}$.

The task’s appeal is timing. A prediction available at commit time reaches the developer who wrote the change while the change is still current, which is the difference between a useful signal and a retrospective one.

2.1.2 The Kamei feature set

Kamei et al. [V] established the modern formulation, together with the change-level metrics that remain standard. The 14 features group into five families:

Family	Features	Intuition
Diffusion	NS, ND, NF, Entropy	A change spread across many subsystems, directories and files is harder to reason about
Size	LA, LD, LT	Large changes carry more opportunity for error
Purpose	FIX	Changes that fix defects are themselves defect-prone
History	NDEV, AGE, NUC	Code touched by many developers, recently, and often
Experience	EXP, REXP, SEXP	Author familiarity with the code being changed

Kamei et al. report approximately **68% accuracy and 64% recall** on defect-inducing changes [V]. These features are used throughout this thesis, taken as published rather than recomputed so that feature extraction cannot differ between the reference labels and the SZZ variants.

2.1.3 Class imbalance and concept drift

Defect-introducing commits are a small minority — **8.54%** on the corpus studied here. Two consequences run through this thesis.

Accuracy is uninformative. A model predicting “clean” for every commit scores 91.5% accuracy on this corpus. §2.6 covers metric selection.

Oversampling is standard, and it is not free. Rebalancing methods increase the influence of minority examples, which is exactly what is wanted when those examples are correct. Chapter 6 shows what happens when they are not.

Projects also change over time — team composition, architecture, process — so the relationship between features and defect-proneness is not stationary. Concept drift in this setting is addressed by Cabral and Minku [V] and subsequent work [C].

2.2 SZZ and its variants

2.2.1 The original algorithm

Śliwerski, Zimmermann and Zeller [C] introduced the procedure now universally called SZZ. Given a commit identified as fixing a defect, it:

1. Extracts the lines modified by the fix;
2. Applies `git blame` (or equivalent) to find, for each line, the most recent prior commit to have modified it;
3. Labels those prior commits defect-introducing.

The algorithm is an inference, and it rests on two premises: that the lines a fix modifies are the lines that were defective, and that the last commit to touch a line is the commit that made it wrong.

2.2.2 The failure taxonomy

Tangled commits violate the first premise. Bug-fixing commits routinely contain reformatting, renaming, comment updates and test changes alongside the fix. Herbold et al. [V], using four annotators per line, found that **between 17% and 32% of all changes in bug-fixing commits address the underlying problem**, rising to **66–87%** when only production-code files are considered; about **11%** of lines produced active annotator disagreement. Every tangled line is blamed, and every commit that last touched one is mislabelled.

Blame attribution error violates the second premise. `git blame` identifies the last commit to *touch* a line, which need not be the commit that made it *wrong*. A later cosmetic edit takes the blame from the change that introduced the fault. This thesis refers to this as the **syntactic line-blame fallacy**, and §2.2.4 explains why it is not measurable in this study’s design.

Structural invisibility. A defect introduced by omission — code that was never written — has no line for blame to find. This mechanism is real in general but, as Chapter 4 explains, is *not available* as an explanation within this thesis’s corpus.

2.2.3 The variants

Each variant adds filtering or selection intended to remove B-SZZ’s over-collection:

Variant	Mechanism	Attribution
B-SZZ	The original	Śliwerski et al. [C]
AG-SZZ	Annotation-graph filtering; excludes cosmetic and format-only changes	Kim et al. [C]
MA-SZZ	Additionally excludes meta-changes (branch changes, property changes)	da Costa et al. [C]
R-SZZ	Retains only the most recent candidate	Davies et al. [C]
L-SZZ	Retains only the largest candidate	Davies et al. [C]
RA-SZZ	Refactoring-aware; removes refactoring-attributable candidates	Neto et al. [C]

The progression assumes that B-SZZ’s dominant error is over-collection and that filtering it out improves the labels. **Chapter 4 tests that assumption directly and finds that for the most aggressive variants, more than half of what the filters remove is correct.**

2.2.4 Developer-informed oracles

Evaluating SZZ requires a reference that does not itself use SZZ. Rosa et al. [V] constructed one by mining commit messages in which developers *explicitly name* the commit that introduced the defect being fixed — yielding **1,930 fix-to-inducing links across eight programming languages**, extended in the journal version to an evaluation of nine variants over **2,304 instances**. Against that oracle, **R-SZZ performed best (F-measure $\approx 61\%$, precision $\approx 66\%$)** while B-SZZ achieved higher recall (≈ 0.69) at markedly lower precision (≈ 0.39) [V].

Two points about this dataset matter for the present work.

First, it **cannot train a JIT-SDP model**. It contains fix-to-inducing links and no clean commits, so there is no negative class and no complete labelling of project history.

Second, **its precision figures are not comparable to this thesis’s without reconciling the denominator**. Rosa et al. score *conditionally* on bug-fixing commits already known to have an inducing commit. This thesis scores over an entire project history of 27,319 commits, the overwhelming majority of which are clean. A variant that is right two times in three when asked “which earlier commit introduced this known defect?” can still be wrong four times in five when asked “is this commit, drawn from all of history, defect-introducing?” **The difference is large enough to read as a contradiction, and Chapter 8 addresses it explicitly.**

2.2.5 JIT-Defects4J and the reference labels

Ni et al. [V] published **JIT-Defects4J**, an extension of LLTC4J [V], comprising **27,319 commits from 21 Java projects, of which 2,332 (8.54%) are labelled defect-introducing**. Its construction is two-stage: lines within bug-fixing commits were annotated manually, retained where at least three participants agreed; `git blame` was then applied to those verified lines.

The labels are therefore `git blame` seeded with human-verified fix lines — not manually verified ground truth for defect-introducing commits. This thesis uses them as its reference precisely because the distinction is tractable: they control for tangling, the dominant false-positive mechanism, while sharing SZZ’s blame step. Comparisons against them isolate the cost of tangled commits, and blame error, being common to both sides, cancels. Chapter 3 §3.2.2 develops the consequences.

2.2.6 Other JIT-SDP datasets

Dataset	Scale	Labels	Relevance here
ApacheJIT [V]	106,674 commits (28,239 bug-inducing), 14 Apache projects	SZZ-derived	Larger, but cannot serve as an oracle for a study of SZZ noise
Defectors [V]	~213K files, 24 Python projects, 18 domains	SZZ-derived	File-level; same objection
ReDef [V]	3,164 defective / 10,268 clean functions, 22 C/C++ projects	Revert-anchored , LLM-triaged, ~92% audited precision	Non-blame labels, but function-level, C/C++, no Kamei features, no fix dates

Scale is not the binding constraint for this thesis; label provenance is. A study measuring what SZZ noise costs cannot use SZZ labels as its reference, which excludes the two largest datasets available.

2.3 Evaluation methodology in defect prediction

2.3.1 Temporal leakage

Random k -fold cross-validation on temporally ordered data allows a model to train on future commits and be tested on past ones. Time-aware evaluation has been advocated repeatedly [C], yet random k -fold remains widespread in the JIT-SDP literature.

Chapter 5 measures the consequence rather than restating the argument. The inflation is +0.137 MCC on a high-capacity model and it scales with model capacity, which means comparisons *between* model families under this protocol are confounded in a direction that favours larger models.

2.3.2 Prequential evaluation

For streams, the standard protocol is *test-then-train*: each example is first predicted, then used for learning. Summarising a metric over a stream requires a forgetting mechanism, conventionally a fading factor applied to the error estimate [C].

An important caveat for this thesis. The prequential-with-fading construction is defined for decomposable losses that can be accumulated example by example. **MCC is not decomposable**, so the construction does not extend to it directly. This thesis therefore reports two ad-hoc summaries of the MCC trajectory — a terminal fading value and a trajectory mean — treats the trajectory mean as primary **on measured variance rather than by appeal to a standard**, and states explicitly that neither is canonical. An earlier version of this work described one of them as “the standard estimator”; that was incorrect and is corrected in Chapter 3 §3.4.2.

2.3.3 Verification latency

Cabral and Minku [V] identified verification latency as a first-class concern in JIT-SDP: labels arrive with a delay, and the delay differs systematically between classes, since a clean commit is only ever *presumed* clean while a defective one must be discovered. Their framing of latency and of the resulting **class imbalance evolution** is the setting this thesis adopts.

On the corpus studied here the median latency is **113 days**, with **53% of defect labels arriving after a 90-day window**. Under a fixed window, a defect label that arrives late is delivered first as a *wrong clean label* and corrected later — so latency is not merely delay but active misinformation, and the misinformation concerns exactly the minority class.

2.4 Online learning for JIT-SDP

Online Bagging [C] adapts bagging to streams by presenting each arriving example to each ensemble member $k \sim \text{Poisson}(\lambda)$ times.

Oversampling Online Bagging (OOB) [C] handles imbalance by setting λ from the running observed class ratio, so minority examples are presented more often as the imbalance grows.

Oversampling Rate Boosting (ORB) [V] extends OOB by monitoring the model’s own recent predictions: when predictions are considerably biased toward one class, the resampling rate of the opposite class is boosted. Cabral and Minku also adopt a safety mechanism intended to avoid training on potentially noisy or outlier minority-class examples [V].

This machinery is the subject of Chapter 6’s central finding. ORB’s λ is a function of the *observed rate* of defect labels — that is, of label **quantity**. It has no access to label **quality**. Chapter 6 measures the coupling directly (Spearman $\rho = -1.000$ between delivered defect-label supply and applied λ) and shows that the mechanism correcting class imbalance is the mechanism that amplifies false positives, hardest exactly when defect labels are scarcest.

Subsequent work addresses concept drift in online JIT-SDP [C] and label noise arising from latency [C]. **The distinction from the present work is the origin of the noise.** Latency-induced noise is a timing artefact: the label is correct but late. SZZ-induced noise is a *labelling* artefact: the label is wrong and will never be corrected. This thesis addresses the second, under conditions that include the first.

2.5 The weak-baseline problem

Deep learning approaches to JIT-SDP — DeepJIT and CC2Vec [C] — reported substantial gains over feature-based models. Two subsequent results complicate that picture.

Zeng et al. [V], evaluating on over **310,370 changes**, found that CC2Vec could not consistently outperform DeepJIT and that neither consistently outperformed traditional feature-based JIT prediction. They then constructed **LApredict** — logistic regression on the added-lines feature alone — which outperformed both deep models while being roughly **81,000–120,000× faster** to train and test [V].

Pornprasit and Tantithamthavorn [V] reported **JITLine**, a simpler feature-based approach that was **26–38% better in F-measure, 17–51% more cost-effective, and 70–100× faster** than CC2Vec and DeepJIT.

Both models are used in this thesis, and the reason is methodological rather than deferential. LApredict has minimal capacity; JITLine has a great deal. Chapter 5 shows that the inflation attributable to evaluation protocol is **+0.137 MCC for JITLine and +0.034 for LApredict**, and that the self-scoring gap is significant for all six SZZ variants under JITLine but only one under LApredict. **Capacity is not incidental to the measurement problem — it is the variable that determines its size**, which offers an alternative reading of why simple baselines have proven so hard to beat.

2.6 Metric selection

Under an 8.54% positive rate, accuracy is uninformative. This thesis uses:

Matthews Correlation Coefficient (MCC) [C] as primary. It incorporates all four confusion-matrix cells and is near zero for any uninformed strategy. A majority-class predictor scores 0.0 MCC and 91.5% accuracy on this corpus.

G-mean, the geometric mean of per-class recalls, which collapses toward zero if either class is abandoned.

Cohen’s κ [C] for agreement between labelling procedures, where the question is concordance rather than prediction quality.

Effort-aware metrics — recall at a fixed inspection budget — arguably match the practitioner’s objective more closely and are identified as future work in Chapter 9, not reported here.

2.7 Learning with noisy labels

Loss correction. Natarajan et al. [C] showed that under class-conditional noise with known rates ρ_0 and ρ_1 , an unbiased estimator of the clean risk can be constructed by reweighting. The correction becomes numerically unstable as $\rho_0 + \rho_1 \rightarrow 1$. **This matters directly here:** Chapter 4 measures $\rho_0 + \rho_1$ approaching 0.8 for the conservative variants, which is why the loss-correction arm evaluated in this thesis is capped.

Sample selection. Co-teaching [C] trains two networks that exchange small-loss examples, on the premise that mislabelled examples exhibit higher loss early in training. It assumes a batch setting with multiple epochs.

Confident Learning [C] estimates the joint distribution of noisy and true labels using model confidences and prunes suspected errors. It is a batch method requiring cross-validated out-of-sample predictions over a fixed dataset.

Why these do not transfer directly. Each assumes a fixed dataset, repeated passes, and labels that are wrong but *stable*. A JIT-SDP stream offers none of these: a single pass, labels arriving late, and labels that are **wrong and then corrected**. Chapter 7 adapts the confidence-based idea to the streaming setting, and its central negative result is

informative about the whole family: a confidence signal cannot distinguish “*this label is wrong*” from “*my model has not learned this pattern yet*”, and the second case is common under severe imbalance with a still-learning model.

2.8 Positioning

Work	Imbalance	Drift	Latency	SZZ noise measured	Online
Kamei et al. [V]	○	○	○	○	○
Cabral & Minku (ORB) [V]	●	●	●	○	●
Rosa et al. (oracle) [V]	○	○	○	● (labels only)	○
Herbold et al. (tangling) [V]	○	○	○	● (labels only)	○
Zeng et al. / Pornprasit et al. [V]	●	○	○	○	○
Natarajan / Co-teaching / CL [C]	●	○	○	◐ (assumed, not measured)	○
This thesis	●	●	●	● downstream cost	●

(● addressed · ◐ partially · ○ not addressed)

The distinguishing cell is the fourth. SZZ evaluation work measures the noise and stops at the labels. Online JIT-SDP work operates under latency but treats label noise as a nuisance to dampen rather than a quantity to measure. Noisy-label learning assumes a noise model rather than estimating one, and assumes a batch setting. This thesis measures **what SZZ-origin noise costs downstream, under latency-aware online evaluation**, and then tests whether that cost can be recovered.

The methodological device that makes this possible is small and worth naming: **an independently constructed reference label set, held out from training and used only for scoring**. Without it, a model trained on SZZ can only be evaluated against SZZ, and the largest effect in this thesis — the +0.230 MCC self-scoring gap — is invisible by construction.

2.9 Citation verification status

[V] — **verified during preparation**. Bibliographic details and the specific attributed claim were checked against the source or publisher record.

Work	Verified detail
Kamei et al., <i>IEEE TSE</i> 39(6):757–773, 2013	Task formulation; ~68% accuracy, 64% recall
Cabral & Minku, <i>ICSE</i> 2019	Class imbalance evolution, verification latency, ORB, safety mechanism
Herbold et al., <i>EMSE</i> , 2022	17–32% of changes / 66–87% production code; four annotators; ~11% disagreement
Ni et al., <i>ESEC/FSE</i> 2022	JIT-Defects4J; 27,319 commits / 2,332 defective / 21 projects; two-stage construction
Rosa et al., <i>ICSE</i> 2021; <i>JSS</i> 202:111729, 2023	1,930 links, 8 languages, 2,304 instances; R-SZZ best ($F \approx 61\%$, $P \approx 66\%$); B-SZZ recall ≈ 0.69 , precision ≈ 0.39 ; PySZZ
Zeng et al., <i>ISSTA</i> 2021	310,370 changes; LAPredict; 81k–120k× speedup
Pornprasit & Tantithamthavorn, <i>MSR</i> 2021, pp. 369–379	JITLine; 26–38% F-measure, 17–51% cost-effectiveness, 70–100× faster
Keshavarz & Nagappan, <i>MSR</i> 2022	ApacheJIT; 106,674 commits, 28,239 bug-inducing, 14 projects
Mahbub, Shuvo & Rahman, <i>MSR</i> 2023	Defectors; ~213K files, 24 Python projects, 18 domains
ReDef (preprint, 2025)	3,164 / 10,268 function-level changes, 22 C/C++ projects, revert-anchored, ~92% precision

[C] — correctly identified, detail not re-verified. Check each against the original before submission:

Śliwerski, Zimmermann & Zeller (original SZZ) · Kim et al. (AG-SZZ) · da Costa et al. (MA-SZZ) · Davies et al. (R-SZZ, L-SZZ) · Neto et al. (RA-SZZ) · time-aware evaluation advocacy (Falessi et al.; Tan et al.) · Gama et al. (prequential evaluation with fading factors) · Oza & Russell (Online Bagging) · Wang, Minku & Yao (OOB) · Cabral & Minku (2023 drift analysis) · Song et al. (2022, latency noise) · Hoang et al. (DeepJIT, CC2Vec) · Matthews (MCC) · Cohen (κ) · Natarajan et al. (loss correction) · Han et al. (Co-teaching) · Northcutt et al. (Confident Learning) · Zhao et al., Zain et al. (JIT-SDP surveys) · Destefanis et al. (methodological audit).

One correction to the project’s own planning documents. An earlier outline attributed to Herbold et al. a finding that “~50%” of SZZ labels are wrong. The verified figures are **17–32% of all changes in bug-fixing commits** addressing the underlying problem, rising to **66–87%** for production-code files only — a different quantity, measured at line rather than commit level. The “~50%” figure does not appear in the source and must not be cited.

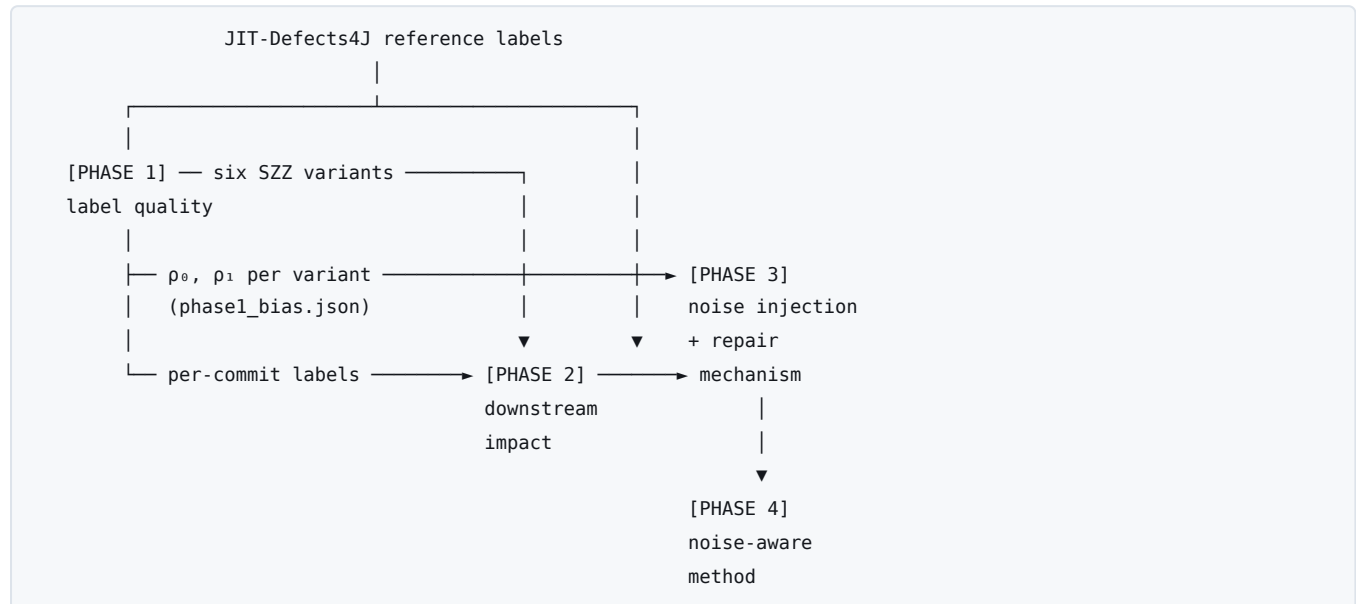
Chapter 3 — Research Design

This chapter states the design of the study as a whole: how the four phases fit together, what data they run on, how the SZZ variants were produced, what is measured and with what statistical protocol, and which validity threats were anticipated in advance rather than discovered afterwards.

The design has one organising commitment. **Every claim is a paired comparison at the project level, and every reported number is regenerable from committed inputs by a named script.** The remainder of this chapter is largely an elaboration of what that commitment required.

3.1 The four-phase pipeline

Each phase consumes the previous phase’s output, and the dependencies are data dependencies rather than narrative ones.



- **Phase 1** measures how far each SZZ variant departs from the reference, and in which direction. Output: per-commit labels, and the flip-rate pair (ρ_0 , ρ_1) per variant.
- **Phase 2** trains models on each label source and evaluates them under four regimes of increasing realism, against two scoring conventions. Output: the inflation ladder and the label-source penalty.
- **Phase 3** uses Phase 1’s flip rates to inject calibrated noise into clean labels, and separately repairs real SZZ labels one error type at a time. Output: which error type costs the learner, and the mechanism.
- **Phase 4** builds and pre-registers a method targeting whatever Phase 3 identifies.

The dependency from Phase 3 to Phase 4 is genuine and was exercised. Phase 4’s original design targeted false negatives. Phase 3 showed that error costs nothing, and Phase 4’s registration was rewritten before its first run. Had the phases been planned in parallel rather than in sequence, that correction could not have happened.

3.2 Data

3.2.1 Corpus

Projects	21 Apache Java repositories
Commits	27,319
Reference defect-introducing	2,332 (8.54%)
Range per project	544–4,026 commits; 19–335 defects; 1.8%–19.3% defect rate
Features	The 14 Kamei change-level metrics

Projects were not selected by us. The corpus is JIT-Defects4J as published, adopted whole so that the selection cannot be tuned to the results. Its consequences for external validity are stated in §3.5 and Chapter 8.

3.2.2 The reference labels

`label_oracle` derives from JIT-Defects4J (Ni et al., ESEC/FSE 2022), an extension of LLTC4J (Herbold et al.). Its provenance was traced through the repository and confirmed against the source paper rather than assumed:

Step	Evidence
<code>label_oracle</code>	= <code>is_buggy_commit</code> in the published feature pickles
The pickles	from the JIT-Fine replication package on Zenodo
Corpus check	the paper’s own table reports 2,332 / 27,319 = 8.54% and matches this corpus project by project

Construction, stated precisely because every downstream claim depends on it. Lines within bug-fixing commits were annotated manually, retained only where at least three participants agreed; `git blame` was then applied to those verified lines to identify the introducing commit; everything else is clean by residual.

The reference is therefore `git blame` **seeded with human-verified fix lines**, not manually verified ground truth for defect-introducing commits. The phrase “developer-verified ground truth” is inaccurate and is not used in this thesis. Chapter 4 §4.1.1 develops the three consequences; the most important is that all measured noise is a **lower bound**, because blame error is present on both sides of every comparison and cancels.

3.2.3 Features and preprocessing

The 14 Kamei metrics are taken as published rather than recomputed, so that feature extraction cannot differ between the reference and the variants. Commits are deduplicated on (project, commit hash) and sorted by author timestamp, which defines the stream order for every temporal regime.

The pipeline was restructured so that two bulky inputs — a 101 MB archive and 830 MB of cloned repositories — are replaced by two small committed derived files (4.2 MB and 0.37 MB). The entire study therefore runs on a continuous-integration runner from committed inputs, which is what makes the reproducibility claim in §3.6 checkable rather than aspirational.

3.2.4 Verification latency

Reconstructing when a defect label would actually have become known requires the author date of each bug-fixing commit. These were extracted once from the cloned repositories and committed as a small table. For each variant, a commit’s arrival time is the date of the fix that blamed it; the union across variants supplies the reference condition’s arrival times.

This is the study’s central construct-validity threat and is declared here rather than in a footnote. The published reference labels carry no native fix-to-inducing linkage, so the reference condition is blame-derived in *construction* and **SZZ-derived in timing**. A coverage sensitivity analysis (67.8% → 100% linkage) bounds part of the exposure; it does not address the timing itself. Every streaming result in Chapters 5–7 inherits this threat.

3.3 SZZ variant implementations

Six variants were produced with **PySZZ v2** (Rosa et al., JSS 2023), the reference implementation released with the developer-informed-oracle study. Using the authors’ own implementation rather than a reimplementing removes one source of unexplained variation, at the cost of inheriting whatever its defaults encode — a trade recorded here as a decision rather than left implicit.

Each variant was run on the same fix-commit set, and its output aligned to the reference over an **identical 27,319-commit denominator**. Partial denominators are a known source of incomparability: a variant emitting fewer candidates can appear more precise merely by being scored on a smaller population.

3.3.1 The label-consistency gate

Mid-project it emerged that the Phase 2 dataset cache had gone stale relative to a Phase 1 label regeneration. Models had been trained on labels one commit older than the noise rates being reported, affecting 1.0–6.5% of commits per variant. The root cause was an unguarded cache keyed on modification time.

The remediation was three-part: a **content-hash staleness guard**, a **provenance sidecar** recording the SHA-256 of every label file used to build the dataset, and an automated **consistency gate** that recomputes the confusion matrix from the training data and asserts it matches the reported noise rates for all six variants. The gate runs in CI **before any compute is spent**, so a mismatch costs two minutes rather than several hours. It was verified in both directions: it fails on the stale data and passes on the corrected data.

This is recorded in the thesis rather than quietly fixed. A study about label-provenance fragility encountered exactly that fragility in its own pipeline, and now ships an executable assertion against its recurrence.

3.4 Metrics and statistical protocol

3.4.1 Metric selection

MCC is primary. At an 8.54% positive rate, accuracy is uninformative: a model predicting “clean” for every commit scores 91.5% accuracy and 0.0 MCC. MCC uses all four confusion-matrix cells and is near zero for any uninformed strategy, which makes it the appropriate primary under this imbalance. **G-mean** is reported alongside as the geometric mean of per-class recalls, since it is sensitive to abandoning the minority class in a way MCC partially shares but expresses differently. **Cohen’s κ** is used for agreement between labelling procedures, where the question is concordance rather than prediction.

3.4.2 The prequential estimator

Under streaming evaluation the metric is a trajectory, not a scalar, and summarising it requires a choice. Two summaries are computed throughout:

- **Terminal fading** — a confusion matrix with exponential forgetting (fading 0.99, effective window ≈ 100 commits), read at end of stream.
- **Time-averaged** — the mean of the MCC trajectory over the stream.

Neither is canonical, and this thesis does not claim otherwise. MCC is not a decomposable loss, so the prequential-with-fading construction does not extend to it directly; of the two, the terminal value is the closer analogue. The time-averaged value is treated as **primary on a measured basis** — roughly half the project-level variance (0.051 against

0.092) — not by appeal to a standard. An earlier version of this work described it as “the standard estimator”, which was wrong and is corrected.

Both are reported for every streaming result. They agree on every comparison in this thesis except one, which is named where it occurs.

3.4.3 The unit of analysis

Every test is paired at the project level, $n = 21$ (or 18 in Phase 4, after held-out exclusion), with seeds averaged before pairing. Seeds control model initialisation and fold randomness; they are not independent observations. Pooling over the full record count would be pseudo-replication, and is the first objection a methodologist would raise.

3.4.4 Effect sizes

Each comparison reports:

- **Wilcoxon signed-rank p** — paired, distribution-free.
- **Matched-pairs rank-biserial correlation** — derived from the same signed ranks as the p -value, so the effect size and the test agree by construction. It reads as *consistency*: +1.000 means every project moved in the same direction.
- **Hodges–Lehmann estimator** — the median of pairwise Walsh averages, the location estimate that accompanies the signed-rank test.
- **Percentile bootstrap confidence interval**, computed **for the Hodges–Lehmann statistic itself**, with the statistic recomputed inside every resample.

Two of these are corrections to an earlier protocol and are recorded as such. Cliff’s δ was initially computed *unpaired*, all-versus-all with denominator $n \cdot m$, discarding the pairing the design exists to preserve; on the headline comparison it reported 0.74 where the paired statistic reports +1.000. Separately, the bootstrap originally resampled the median while the Hodges–Lehmann estimate was reported beside it — an interval for a different quantity than the point estimate.

3.4.5 Multiplicity

Seventy-four tests are run in Phase 2 alone. Multiplicity is corrected **twice**:

- **Holm within test family** — families grouped by the comparison being made.
- **Holm globally across all tests** — no family argument required.

The global column exists because the families were defined *during* analysis, not declared in advance. A sceptic can reasonably argue the grouping was chosen to help the results; **a claim surviving global Holm needs no defence of how families were drawn**. Every headline claim in this thesis survives the global column, and where a claim survives only within-family correction, that is stated.

3.4.6 Pre-registration

Phases 1–3 are exploratory. Their hypotheses were formed during analysis and every result is labelled accordingly; the global multiplicity correction is the sensitivity analysis that makes them defensible without a pre-registration.

Phase 4 alone is pre-registered, with hypotheses, primary model, acceptance bar and correction committed to version control before its first run. Its registration was revised once, after Phase 3 refuted the hypothesis it was built on and while Phase 4 still had zero results — the only point at which revision is legitimate. Chapter 7 §7.1 gives the full account.

3.5 Threats to validity, framed in advance

Stating the framework before the results appear is deliberate: it prevents the threat list from being assembled to fit whatever was found.

Construct validity. The reference is `git blame` seeded with verified fix lines, not ground truth, so comparisons isolate tangling rather than total labelling error and measured noise is a lower bound. Arrival times for the reference condition are SZZ-derived (§3.2.4) — the single most serious threat in the study.

Internal validity. Temporal leakage is the threat the design exists to expose, and is therefore a *manipulated variable* rather than a defect. Cache staleness is addressed by the gate (§3.3.1). Repair and injection experiments are oracle-assisted by construction and are labelled as upper bounds, not methods.

External validity. One benchmark family: 21 Apache Java repositories of a single dataset lineage. The corpus is SZZ-shaped, since changes adding no new lines were excluded by the dataset authors, so defects introduced purely by deletion cannot appear. Projects are weighted equally despite an eighteen-fold range in defect count.

Conclusion validity. With 21 paired projects, effects below roughly 0.04 MCC are not resolvable — a limit that becomes load-bearing for the latency decomposition, which is reported as unresolved rather than as null. Multiplicity is corrected twice. The number of exploratory comparisons is large, which is precisely why the global correction is reported.

One scope limit declared before Phase 3 ran. Because the reference labels are themselves blame output, every reference positive is blame-reachable by construction. **No false negative measured against this reference can be attributed to blame’s structural inability to trace a defect.** Phase 3 can therefore study false-negative noise arising from over-filtering and seed-line divergence — the two mechanisms Chapter 4 shows are actually operating — but can make no claim about blame-unreachability. Declaring this in advance prevents an unavailable explanation from being offered afterwards.

3.6 Reproducibility

Every number in Chapters 4–7 is regenerable from committed inputs by a named script, and the chapter-to-artefact mapping is explicit:

Chapter	Script	Principal outputs
4	<code>experiments/evaluate_confusion_matrix.py</code>	quality table, κ matrix, <code>phase1_bias.json</code>
5	<code>experiments/run_phase2_impact.py</code>	regime $\times$ label-source matrix, 74 tests
6	<code>experiments/run_phase3_noise.py</code> , <code>run_phase3_addendum.py</code>	dose curves, repair statistics, λ traces
7	<code>experiments/run_phase4_tuning.py</code> , <code>run_phase4_na_orb.py</code>	frozen configuration, registered tests
Appendix	<code>experiments/run_robustness_checks.py</code>	project table, floor sensitivity, mechanism test

Runtime dependencies are pinned to the exact versions every result was computed with, so a runner reproduces the local environment rather than approximating it. The label-consistency gate runs before every experiment in CI.

The next four chapters report what these scripts produce.

Chapter 4 — Phase 1: Label Disagreement and Oracle Comparison

This chapter answers **RQ1**: how much do SZZ variants disagree with one another and with an independently constructed reference, and what is the direction of each variant’s labelling bias?

The chapter is deliberately placed before any predictive modelling. Everything in Chapters 5 to 7 is conditioned on the noise characterised here: the flip rates estimated in §4.3 are the parameters injected in Chapter 6, and the reference labels defined in §4.1 are the scoring key used throughout Chapter 5.

4.1 Method

4.1.1 The reference condition, and what it is not

Every comparison in this thesis is made against `label_oracle`, taken from JIT-Defects4J (Ni et al., ESEC/FSE 2022), itself an extension of LLTC4J (Herbold et al.). Because the interpretation of every subsequent result depends on exactly what these labels are, we state their construction precisely.

The dataset authors describe a two-stage procedure. First, within each bug-fixing commit, individual *lines* were annotated as contributing to the fix, with a line retained only where **at least three participants assigned it the same label**. Second, for each such verified fix line, **git blame was used to identify the commit that introduced it**. Commits not flagged by this procedure are treated as clean by residual.

Three consequences follow, and they constrain the claims this thesis is entitled to make.

The reference is not manually verified ground truth for defect-introducing commits. Human verification applies to the *fix lines*. The step from a fix line to an introducing commit is algorithmic. The correct description — used consistently throughout this thesis — is **git blame seeded with human-verified fix lines**. The phrase “developer-verified ground truth” is inaccurate and is not used.

The comparison therefore isolates tangling, not total labelling error. SZZ blames *every* line modified in a bug-fixing commit, including refactoring, formatting and comment changes, then applies filters to remove what it over-collected. The reference blames *only* the lines three annotators agreed were fixing the bug. Both then apply the same blame step. What separates them is exactly one thing: knowledge of which lines in a fix actually fix the bug. That is the definition of a tangled commit, and it is what these comparisons measure.

All measured noise is consequently a lower bound. Any systematic error in `git blame` — the syntactic line-blame fallacy by which blame attributes a line to the last commit that touched it rather than the one that made it wrong — is present on *both* sides of every comparison and cancels. Noise measured against genuine ground truth would be larger by an unknown amount.

This is a weaker claim than “we compare SZZ against truth”, and a more precise one. It is also the claim the data actually support.

4.1.2 Variants, corpus and alignment

Six SZZ variants were executed using PySZZ v2 (Rosa et al., JSS 2023), the reference implementation released with the developer-informed-oracle study:

Variant	Refinement over its predecessor
B-SZZ	The original: blame every line modified in the fix
AG-SZZ	Annotation-graph filtering; excludes cosmetic and format-only changes
MA-SZZ	Additionally excludes meta-changes such as branch and property changes
R-SZZ	Of the candidate inducing commits, retains only the most recent
L-SZZ	Retains only the largest candidate
RA-SZZ	Refactoring-aware: removes candidates attributable to refactorings

The corpus is 21 Apache Java projects, 27,319 commits, of which 2,332 (**8.54%**) carry a reference defect label. Each variant’s output was aligned to the reference over an **identical 27,319-commit denominator**, so that precision and recall are computed against the same population in every row. This matters: partial denominators are a common source of incomparability in the SZZ literature, since a variant that emits fewer candidates can appear more precise merely by being evaluated on a smaller set.

Per-variant quality is reported as precision, recall, F1, MCC and Cohen’s κ , together with the two flip rates defined in §4.3. Inter-variant agreement is reported as pairwise Cohen’s κ .

4.2 Results

4.2.1 Per-variant quality against the reference

Variant	Precision	Recall	F1	ρ_0 (false-alarm)	ρ_1 (miss)	MCC	κ	TP	FP	FN	TN
B-SZZ	0.186	0.641	0.288	0.263	0.359	0.232	0.179	1,495	6,565	837	18,422
AG-SZZ	0.186	0.467	0.266	0.192	0.533	0.188	0.163	1,090	4,786	1,242	20,201
MA-SZZ	0.183	0.482	0.265	0.201	0.518	0.188	0.161	1,124	5,030	1,208	19,957
L-SZZ	0.272	0.267	0.269	0.067	0.733	0.202	0.202	622	1,665	1,710	23,322
R-SZZ	0.232	0.300	0.262	0.093	0.700	0.185	0.183	699	2,316	1,633	22,671
RA-SZZ	0.184	0.438	0.259	0.181	0.562	0.178	0.158	1,022	4,531	1,310	20,456

Two observations dominate.

A precision ceiling at 27.2%. No variant exceeds it. Between 72.8% and 81.5% of every commit any variant flags as defect-introducing is not one, judged against the reference. The two decades of refinement separating B-SZZ from RA-SZZ have not produced a variant whose positive predictions are more often right than wrong.

Noise is asymmetric, and the asymmetry differs by variant. B-SZZ is false-positive-heavy: it flags 8,060 commits to capture 1,495 real ones, achieving the highest recall (0.641) at a false-alarm rate of 26.3%. L-SZZ and R-SZZ are the mirror image: false-alarm rates of 6.7% and 9.3%, but they miss 73.3% and 70.0% of real defect-introducing commits respectively. The annotation-graph family sits between.

This asymmetry is the single most consequential fact in the thesis. **Modelling SZZ noise as symmetric random label flips — the standard assumption in the learning-with-noisy-labels literature — misrepresents it entirely.** A variant that misses seven defects in ten and one that raises four false alarms for every true one are different problems, and Chapter 6 shows they have different downstream consequences.

No variant dominates. Ranking by F1 compresses the differences into a band of 0.259–0.288 and hides the trade-off completely. Ranking by MCC puts B-SZZ first (0.232) and RA-SZZ — the most refined variant — last (0.178). Ranking by κ reverses the top: L-SZZ first (0.202), B-SZZ fourth. **The choice of summary metric determines which variant appears best,** which is itself a finding about how SZZ variants are compared in the literature.

4.2.2 Inter-variant agreement

Pairwise Cohen’s κ between the six variants:

	B-SZZ	AG-SZZ	MA-SZZ	L-SZZ	R-SZZ	RA-SZZ
B-SZZ	—	0.690	0.695	0.330	0.427	0.630
AG-SZZ	0.690	—	0.933	0.467	0.587	0.859
MA-SZZ	0.695	0.933	—	0.477	0.596	0.883
L-SZZ	0.330	0.467	0.477	—	0.636	0.477
R-SZZ	0.427	0.587	0.596	0.636	—	0.566
RA-SZZ	0.630	0.859	0.883	0.477	0.566	—

Set against the agreement of each variant with the reference — **κ between 0.158 and 0.202** — the structure is stark. The annotation-graph family agrees with itself at κ 0.859–0.933, which conventional interpretation calls *almost perfect*. The same variants agree with the reference at $\kappa \approx 0.16$, which the same convention calls *slight*.

Variants agree with each other four to six times more strongly than any of them agrees with the reference. They are not independent estimators converging on a common truth. They share a blame step, a set of assumptions, and therefore a set of failure modes.

The methodological implication is direct and should be stated in any paper that uses SZZ. **High inter-tool agreement has been treated in the literature as evidence of correctness. On this corpus it is evidence of shared error.** Two procedures that make the same mistakes will agree precisely where both are wrong.

4.2.3 Per-project variance

Aggregate figures conceal substantial per-project spread:

Variant	Precision (min / median / max)	Recall (min / median / max)
B-SZZ	0.056 / 0.199 / 0.386	0.220 / 0.700 / 0.919
AG-SZZ	0.053 / 0.222 / 0.325	0.074 / 0.389 / 0.819
MA-SZZ	0.053 / 0.188 / 0.325	0.098 / 0.417 / 0.819
L-SZZ	0.072 / 0.271 / 0.491	0.080 / 0.245 / 0.432
R-SZZ	0.058 / 0.213 / 0.450	0.061 / 0.250 / 0.552
RA-SZZ	0.042 / 0.213 / 0.328	0.110 / 0.361 / 0.789

B-SZZ’s recall ranges from 0.220 to 0.919 across projects — a span of seven-tenths. AG-SZZ’s ranges from 0.074 to 0.819. **A single corpus-level noise rate is a poor description of any individual project**, which is why every statistical test in this thesis is paired at the project level rather than pooled over commits, and why Chapter 6 injects noise per project rather than globally.

4.3 The bias model

Phase 3 requires a parametric description of each variant’s noise. We adopt the standard class-conditional formulation, in which a variant is treated as a noisy channel applied to the reference labels:

- ρ_0 = P(variant labels a commit defective | the reference labels it clean) — the **false-alarm rate**
- ρ_1 = P(variant labels a commit clean | the reference labels it defective) — the **miss rate**

Variant	ρ_0	ρ_1	Character
B-SZZ	0.263	0.359	False-positive-heavy
MA-SZZ	0.201	0.518	Intermediate
AG-SZZ	0.192	0.533	Intermediate
RA-SZZ	0.181	0.562	Intermediate
R-SZZ	0.093	0.700	False-negative-heavy
L-SZZ	0.067	0.733	False-negative-heavy

These six pairs are serialised to `phase1_bias.json` and are the calibration points for Chapter 6’s injection profiles: B-SZZ supplies the FP-heavy profile, RA-SZZ the intermediate profile, and L-SZZ the FN-heavy profile.

Two properties of this channel are worth stating explicitly.

First, $\rho_0 + \rho_1$ approaches 0.8 for the conservative variants. Loss-correction methods that invert the noise channel become numerically unstable as $\rho_0 + \rho_1 \rightarrow 1$, which is why the correction arm evaluated in Chapter 7 is capped.

Second, because the positive class is only 8.54% of the corpus, **equal flip rates do not imply equal numbers of flipped labels**. B-SZZ’s ρ_0 of 0.263 applied to 24,987 clean commits produces 6,565 false positives; L-SZZ’s ρ_1 of 0.733 applied to 2,332 defective commits produces 1,710 false negatives. This asymmetry between *rate* and *volume* is not a technicality — Chapter 6 shows that it is the distinction on which the central mechanistic finding of this thesis turns.

4.3.1 Where the false negatives come from

The conservative variants miss 70–73% of reference-labelled defects. A natural explanation is structural: that line-tracking heuristics cannot blame code which was never written, so defects introduced by omission are invisible.

That explanation is unavailable here. Because the reference labels are themselves blame output (§4.1.1), every reference positive is blame-reachable from some fix line *by construction*. No false negative measured against this reference can be caused by blame being structurally unable to trace a defect. The limitation is a property of the corpus and must be declared, not assumed away.

Decomposing each variant’s false negatives against B-SZZ, the most permissive variant, gives the measured alternative:

Variant	Total FN	Also missed by B-SZZ	Found by B-SZZ, then filtered away
AG-SZZ	1,242	765 (62%)	477 (38%)
MA-SZZ	1,208	763 (63%)	445 (37%)
RA-SZZ	1,310	765 (58%)	545 (42%)
R-SZZ	1,633	808 (49%)	825 (51%)
L-SZZ	1,710	807 (47%)	903 (53%)

Two mechanisms of comparable weight:

1. **Seed-line divergence (47–63%).** Even B-SZZ, which blames every modified line, misses 837 reference positives. Both procedures run `git blame`; they disagree because they blame *different lines*. B-SZZ traces from all modified lines, the reference from the subset three annotators verified. The two land on different commits.
2. **Self-inflicted filtering (37–53%).** Commits B-SZZ identified correctly, which the variant’s own refinement then discarded. For the two most conservative variants this is the *larger* component: R-SZZ discards 55.2% and L-SZZ 60.4% of the reference defects B-SZZ had already found.

The refinements are the problem, not the blame step. Each variant was designed to remove B-SZZ’s false positives, and each succeeds — ρ_0 falls from 0.263 to 0.067. But more than half of what the most aggressive filters remove is correct. This is a precision/recall trade made badly, and it is measurable precisely *because* both sides share a blame step.

The variants are not strict subsets of B-SZZ: between 4.7% (R-SZZ) and 11.9% (RA-SZZ) of their flags lie outside it, so they differ in candidate-selection strategy as well as in filtering.

4.4 Discussion — answering RQ1

How much do the variants disagree with each other? Substantially, and structurally. The annotation-graph family (AG/MA/RA) forms a tight cluster at κ 0.859–0.933. L-SZZ and R-SZZ form a second, looser cluster (κ 0.636). Agreement *between* clusters falls to κ 0.330–0.596. The variants partition into two design philosophies — filter the candidate set, or select a single candidate from it — and the partition is visible in the agreement matrix.

How much do they disagree with the reference? Far more than with each other. κ 0.158–0.202 against the reference versus up to 0.933 among themselves. This is the chapter’s central negative result and it undermines a common methodological practice: cross-variant agreement cannot be used to argue that a labelling is correct.

What is the direction of each variant’s bias? Bifurcated, and predictable from the variant’s design. B-SZZ over-flags because it blames indiscriminately. L-SZZ and R-SZZ under-flag because each collapses a candidate set to a single commit, discarding the rest. The annotation-graph family occupies the middle because it filters without collapsing. **Refinement has not reduced error; it has relocated error from the false-positive column to the false-negative column.**

That relocation is defensible only if false positives are more costly than false negatives to whatever consumes the labels. Chapter 6 tests that premise directly, and finds the answer is not the one the variants’ designers appear to have assumed.

4.4.1 What this chapter licenses, and what it does not

Licensed. That SZZ labels on this corpus are noisy at rates far exceeding those usually assumed; that the noise is class-conditional rather than symmetric; that variants share failure modes; that measured noise is a lower bound on true noise.

Not licensed. Any statement that a particular variant is “wrong” in an absolute sense — the reference is a blame-derived construct, not truth. Any generalisation beyond Apache Java projects of this dataset lineage. Any claim about defects introduced purely by deletion, which the corpus excludes by construction.

The next chapter asks the question Phase 1 cannot: whether noise of this magnitude and shape actually changes what a defect prediction model appears to achieve.

Chapter 5 — Phase 2: Downstream Impact Under Honest Evaluation

Chapter 4 established that SZZ labels are noisy, that the noise is class-conditional, and that variants share failure modes. None of that tells us whether the noise *matters*. A model might be robust to it; a metric might be insensitive to it; the effect might be smaller than the variance between projects.

This chapter answers **RQ2**: how does the choice of SZZ label source shift measured JIT-SDP performance across evaluation regimes of increasing realism?

Its central finding is not the one the research question anticipates. The label source matters, but **the evaluation protocol matters more**, and the largest single effect in the entire study is an artefact of how the field scores its models rather than of which labels it trains them on.

5.1 Method

5.1.1 The design

Phase 2 is a four-way factorial grid.

Dimension	Levels
Model	LApredict, JITLine, ORB
Training label source	Reference labels + six SZZ variants (7)
Evaluation regime	Naive k -fold, chronological, chronological-online, prequential-with-latency (4)
Scoring convention	Oracle-scored, self-scored (2)

Twenty-one projects, ten random seeds, giving **16,380 evaluation records**.

Models. LApredict is logistic regression on a single feature — lines added — included because prior work has found such baselines competitive with deep models, making it a test of whether measured effects are properties of the task or of model capacity. JITLine is a 100-tree random forest with SMOTE oversampling and threshold moving, representing the high-capacity batch state of the practice. ORB is a 20-member online logistic ensemble with Poisson oversampling whose rate responds to the model’s recent prediction bias; it is the only model of the three that can operate on a stream.

Regimes, in increasing order of realism:

1. **Naive k -fold** — random 10-fold cross-validation, ignoring commit order. Included precisely because it is indefensible: a model may train on 2020 commits and be tested on 2015 ones. It is the regime most widely used in the JIT-SDP literature, which is why it is the baseline rather than the strawman.
2. **Chronological** — a single 50/50 split in commit-timestamp order. Removes temporal leakage; retains the assumption that all training labels are available at training time.
3. **Chronological-online** — ORB trained sequentially on the first half, then frozen and evaluated on the second. Isolates the learner change from the regime change.
4. **Prequential with verification latency** — test-then-train streaming. Each commit is predicted when it arrives, and its label is delivered only when it would actually have become known.

5.1.2 The scoring convention, and why it is the experiment

The distinction between *oracle-scored* and *self-scored* is the conceptual core of this chapter.

- **Self-scored** — the model is evaluated against the same noisy SZZ labels it was trained on. **This is what the JIT-SDP literature does.**
- **Oracle-scored** — the model is trained on SZZ labels but evaluated against the independently constructed reference labels, held out and never used in training.

Self-scoring measures how faithfully a model reproduces a heuristic. Oracle-scoring measures how well it identifies what the reference calls defect-introducing. These are different quantities, and the field reports the first while interpreting it as the second. Holding out an independently sourced scoring key makes the gap between them measurable, and measuring that gap is the purpose of Phase 2.

5.1.3 Verification latency

Under the prequential regime, a defect label is not available when the commit arrives; it becomes available when the defect is found and fixed. The timeline implemented is:

- At commit time t : the model predicts; the prediction is scored immediately against the reference label.
- If the training label is clean: delivered as a training example at $t + W$, with $W = 90$ days.
- If defective and the fix arrives within the window: delivered as a positive at the fix time.

- If defective but the fix arrives later: delivered **first as a negative at $t + W$, then corrected to positive at the fix time.**
- If defective with no linked fix: delivered as a negative at $t + W$ and never corrected.

The third case is what makes latency more than a delay. A model operating under a 90-day window does not merely wait for labels — **it is actively trained on labels it will later learn were wrong.** On this corpus, 53.0% of defect labels arrive after the window, with a median latency of 113 days.

5.1.4 Statistical protocol

Every test is paired at the **project** level, $n = 21$, with seeds averaged before pairing. Pooling over the 16,380 records would be pseudo-replication: the seeds are not independent observations of anything.

Each comparison reports a Wilcoxon signed-rank p -value, a **matched-pairs rank-biserial correlation** (the proportion of projects moving in the effect’s direction, derived from the same signed ranks as the p -value), a **Hodges–Lehmann** location estimate, and a percentile bootstrap confidence interval computed for the Hodges–Lehmann statistic itself.

Seventy-four tests were run. Multiplicity is corrected twice: **within test family**, and **globally across all 74**. The global column exists because the families were defined during analysis rather than declared in advance, so a sceptic could argue the grouping was chosen to favour the results. A claim surviving global Holm needs no defence of how families were drawn. Thirty-two tests survive within-family correction; **twenty-one survive global correction**, and it is the global column that the headline claims rest on.

Phases 1–3 are exploratory and are labelled as such throughout. Only Phase 4 is pre-registered.

5.2 Results

5.2.1 The full matrix

Mean MCC over 21 projects $\times$ 10 seeds.

Batch models

Model	Labels	<i>k</i> -fold oracle-scored	<i>k</i> -fold self-scored	Chrono oracle-scored	Chrono self-scored
JITLine	reference	0.2430	—	0.1016	—
JITLine	B-SZZ	0.1751	0.4129	0.1324	0.2232
JITLine	AG-SZZ	0.1164	0.3037	0.0904	0.1351
JITLine	MA-SZZ	0.1272	0.3295	0.0786	0.1395
JITLine	L-SZZ	0.1391	0.2434	0.0804	0.1482
JITLine	R-SZZ	0.1131	0.1801	0.0626	0.0920
JITLine	RA-SZZ	0.1164	0.3070	0.0857	0.1402
LApredict	reference	0.2058	—	0.1734	—
LApredict	B-SZZ	0.1941	0.3510	0.1634	0.2918
LApredict	AG-SZZ	0.2003	0.2472	0.1685	0.1975
LApredict	MA-SZZ	0.1995	0.2626	0.1664	0.2003
LApredict	L-SZZ	0.2078	0.2679	0.1706	0.2174
LApredict	R-SZZ	0.2016	0.1779	0.1731	0.1666
LApredict	RA-SZZ	0.2000	0.2603	0.1679	0.2143

Reference-trained cells have no separate self-scored value: the two conventions coincide by construction.

Online model (ORB)

Labels	Chrono-online oracle	Chrono-online self	Prequential (time-avg)	Prequential (terminal)
reference	0.0777	—	0.0970	0.0685
B-SZZ	0.0767	0.1613	0.0578	0.0566
L-SZZ	0.0082	0.0695	0.0582	0.0209
RA-SZZ	0.0193	0.0673	0.0395	0.0184
AG-SZZ	0.0177	0.0684	0.0365	0.0125
MA-SZZ	0.0216	0.0717	0.0353	−0.0030
R-SZZ	0.0114	0.0349	0.0315	0.0104

Two features of this table are worth marking before the analysis.

First, **LApredict** — **logistic regression on one feature** — **is the best model in the corpus under a chronological split** (0.1734 against JITLine’s 0.1016 on reference labels). The random forest’s advantage exists only under *k*-fold.

Second, MA-SZZ’s apparently harmful negative MCC (−0.0030) exists only under the terminal estimator; under the time-averaged estimator it is +0.0353. **No MCC is quoted anywhere in this thesis without naming the estimator**, and §5.2.5 explains why.

5.2.2 The inflation ladder

Step	Configuration	MCC	What was removed
0	JITLine, B-SZZ labels, k -fold, self-scored	0.4129	— (the literature’s configuration)
1	JITLine, B-SZZ labels, k -fold, oracle-scored	0.1751	Circular self-scoring (−0.238)
2	JITLine, reference labels, chronological	0.1016	Temporal leakage
3	ORB, reference labels, prequential + latency	0.0970	Batch-to-stream

A number that reads as 0.41 under the field’s standard protocol is 0.10 under a defensible one.

The ladder must be read as **descriptive, not causal**. Step 3 changes the learner, the regime and the evaluation window simultaneously. §5.3 explains why an earlier attempt to decompose it was withdrawn.

5.2.3 Result 1 — Temporal leakage inflates, in proportion to model capacity

Random k -fold against a chronological split, paired by project. Fourteen comparisons: seven label sources × two batch models.

Model	Labels	k -fold	Chrono	HL	95% CI	Rank-biserial	Holm (family)	Holm (global)
JITLine	reference	0.2430	0.1016	+0.1373	[+0.1046, +0.1721]	+1.000	1.3e-05	7.1e-05
JITLine	L-SZZ	0.1391	0.0804	+0.0667	[+0.0216, +0.0900]	+0.792	0.0094	0.0389
JITLine	R-SZZ	0.1131	0.0626	+0.0515	[+0.0258, +0.0764]	+0.706	0.0393	0.1607
LApredict	reference	0.2058	0.1734	+0.0339	[+0.0006, +0.0647]	+0.489	0.4015	1.000

(Four of fourteen shown; the remainder follow the same pattern.)

The headline is JITLine on reference labels: **+0.1373 MCC, with a rank-biserial of exactly +1.000 — all 21 projects, without exception**. The exact test saturates.

The effect scales with model capacity. JITLine loses 0.137; LApredict loses 0.034 and does not survive correction. A single-feature logistic regression has little capacity to exploit temporal leakage; a 100-tree forest has a great deal.

This is the most damaging form the finding could take. The inflation is largest exactly where the field reports its strongest results, so **the apparent superiority of high-capacity models over simple baselines is partly an artefact of the evaluation protocol that measures it.**

5.2.4 Result 2 — Self-scoring inflates further, and it is model-dependent

Thirty-six comparisons: for each (model, label source, regime), the same predictions scored against the training labels versus against the reference.

Model	Labels	Regime	Self	Oracle	HL	95% CI	Rank-biserial	Holm (global)
JITLine	B-SZZ	k-fold	0.4129	0.1751	+0.2303	[+0.1886, +0.2784]	+0.991	1.0e-04
JITLine	MA-SZZ	k-fold	0.3295	0.1272	+0.1985	[+0.1393, +0.2648]	+1.000	1.0e-04
JITLine	RA-SZZ	k-fold	0.3070	0.1164	+0.1861	[+0.1221, +0.2523]	+1.000	1.0e-04
JITLine	AG-SZZ	k-fold	0.3037	0.1164	+0.1797	[+0.1201, +0.2456]	+1.000	1.0e-04
LApredict	B-SZZ	k-fold	0.3510	0.1941	+0.1549	[+0.1213, +0.2029]	+0.957	6.0e-04
LApredict	AG-SZZ	k-fold	0.2472	0.2003	+0.0510	[-0.0123, +0.1100]	+0.351	1.000
LApredict	R-SZZ	k-fold	0.1779	0.2016	-0.0238	[-0.0662, +0.0265]	-0.221	1.000

+0.2303 MCC is the largest effect in this thesis. It is also the effect with the clearest methodological reading: a random forest trained on B-SZZ labels appears roughly two and a half times better at finding defects than it is, because it is being graded by the heuristic that taught it.

The gap is model-dependent, and the pattern is diagnostic. Under JITLine it is large and significant for every one of the six variants. Under LApredict it is significant only for B-SZZ, and for R-SZZ it is *negative*. A single-feature logistic regression cannot memorise a heuristic’s idiosyncrasies; a high-capacity forest can. **The self-scoring gap is a capacity phenomenon, which is exactly what one would predict if the mechanism is a model fitting the error structure of its labels.**

The mechanism, tested directly. That reading was, until recently, an interpretation the gap was *consistent with* rather than one it established. We therefore tested it: restricting to the 8,060 B-SZZ-flagged commits and asking whether a model can separate B-SZZ’s false positives from its true positives using the same 14 features.

	Mean AUC
Random forest, chronological split per project	0.599
Same pipeline, training labels permuted (the null)	0.514

Paired across projects: **HL +0.088, CI [+0.041, +0.131], 17/21 projects, $p = 0.0016$.**

SZZ’s errors carry structure the features can detect, so a model *can* fit them. The mechanism is real. **It is also weak — an AUC of 0.60 is modest separability — and the +0.230 gap is almost certainly not explained by this structure alone.** The claim this thesis makes is “the error structure is learnable”, not “SZZ’s mistakes are easy to predict”.

5.2.5 Result 3 — Under realistic streaming, the label source matters

Under the prequential regime with real verification latency, ORB trained on reference labels against ORB trained on each variant, all scored against the reference.

Reference vs	Mean gap	HL	95% CI	Rank-biserial	Projects	Holm (family)	Holm (global)
R-SZZ	+0.0655	+0.0603	[+0.0418, +0.0816]	+0.948	19/21	0.0001	0.0009
MA-SZZ	+0.0617	+0.0638	[+0.0390, +0.0845]	+0.896	18/21	0.0003	0.0042
AG-SZZ	+0.0605	+0.0600	[+0.0290, +0.0851]	+0.818	17/21	0.0009	0.0243
RA-SZZ	+0.0575	+0.0577	[+0.0357, +0.0808]	+0.879	18/21	0.0004	0.0064
B-SZZ	+0.0392	+0.0412	[+0.0219, +0.0521]	+0.879	18/21	0.0004	0.0064
L-SZZ	+0.0388	+0.0341	[+0.0140, +0.0552]	+0.723	16/21	0.0025	0.1241

All six intervals exclude zero; **five of six survive global Holm correction.**

The B-SZZ comparison is the most informative row. Because the reference and B-SZZ share the same blame step and differ only in which lines seed it (§4.1.1), this contrast is a near-controlled ablation of tangled commits. Its value is **+0.041 MCC, CI [+0.022, +0.052]** — a defensible estimate of what knowing which lines in a fix actually fix the bug is worth to a deployed online learner.

The gap does not track recall. The correlation between a variant’s oracle-versus-variant gap and its recall is -0.18 . L-SZZ (recall 0.267) and B-SZZ (recall 0.641) have almost identical gaps. Whatever drives the penalty, it is not simply how many defects the labeller finds — a point Chapter 6 returns to and resolves.

Estimator dependence, reported rather than buried. Two summaries of the prequential MCC trajectory are available: a *terminal* value computed from a fading confusion matrix (effective window ≈ 100 commits), and the *time-averaged* mean of the trajectory. They agree on every comparison in this thesis except one: under the terminal estimator the B-SZZ row alone loses significance. The time-averaged estimator is treated as primary on a measured basis — roughly half the project-level variance (0.051 against 0.092) — and **not** by appeal to a standard. MCC is not a decomposable loss, so the prequential-with-fading construction does not extend to it directly; both summaries are ad-hoc, and this thesis says so rather than claiming canonical status for either.

5.3 An honest negative: the batch-to-stream contrast could not be isolated

An earlier version of this work reported that verification latency accounts for roughly 9% of the drop from batch to streaming performance. **That claim has been withdrawn, and the withdrawal is reported here rather than removed.**

The comparison was not identified. It changed three things at once: the learner (forest $\rightarrow$ online ensemble), the label timing (immediate $\rightarrow$ delayed), and the evaluation window (a held-out second half $\rightarrow$ the whole stream). Any of the three could produce the observed difference.

A full 2×2 factorial — learner (frozen / adaptive) crossed with labels (immediate / delayed) — was built to isolate the effects:

Contrast	Estimate	Interval
Latency effect	$\approx +0.023$	contains zero
Adaptivity effect	≈ -0.046	contains zero
Interaction	—	contains zero

Every contrast, including the interaction, has a confidence interval spanning zero. With 21 paired projects, effects below roughly 0.04 MCC are not resolvable, and these fall in that range.

The honest conclusion is that **21 projects cannot resolve the decomposition**, and it is reported as unresolved. Chapter 6 returns to the latency question with a design that holds the learner fixed by construction, and there the question does become answerable.

5.4 Robustness

Estimator and fading factor. The label-source conclusion was re-examined across fading factors 0.90–0.999 (effective windows 10–1000 commits) crossed with warm-up skips of 0/5/10/25% — 20 settings $\times$ 6 variants = 120 comparisons. In **120 of 120**, the Hodges–Lehmann estimate favours the reference labels, the bootstrap interval excludes zero, and the comparison survives *grid-wide* Holm correction.

This is one result re-examined under different analytical choices, not 120 findings, and grid-wide correction is applied so the framing cannot be mistaken.

Project weighting. Every figure is an unweighted mean over projects carrying between 19 and 335 reference defects; eight of 21 carry fewer than 25 in the chronological test half. Re-running the headline contrasts with the smallest projects excluded:

Claim	All 21	≥ 25 defects ($n = 13$)
k -fold inflation	+0.137	+0.136
Self-scoring gap (k -fold)	+0.230	+0.232
Label source (reference vs B-SZZ)	+0.041	+0.043

No estimate moves by more than 0.002 MCC and all three still survive correction. **The conclusions are not artefacts of the small projects.** One claim does weaken: the self-scoring gap under the *chronological* regime falls from +0.098 to +0.075 and loses significance — not the headline, but the deployment-relevant regime, and reported here rather than omitted.

5.5 Discussion — answering RQ2

The label source shifts measured performance, and the shift is significant under every regime that permits a fair test. Under realistic streaming the penalty for training on SZZ rather than the reference is 0.034–0.064 MCC, against a deployable baseline of roughly 0.097 — between a third and two-thirds of the available signal.

But the regime shifts it more. Moving from k -fold to chronological costs JITLine 0.137. Removing circular scoring costs a further 0.230. **The two protocol effects together are roughly nine times the label-source effect.**

This inverts the question the chapter set out to ask. Phase 2 was designed to measure what SZZ noise costs. What it found is that **the cost of SZZ noise is small relative to the cost of the field’s evaluation conventions** — and that the two interact, because self-scoring is only possible *because* the labels are heuristic. If the field had ground truth, it would score against it, and the +0.230 gap could not exist.

The implication for reading published results. A JIT-SDP result reported under random k -fold with self-scored SZZ labels is not a measurement of defect-prediction capability. On this corpus the difference between that protocol and a defensible one is 0.41 versus 0.10 MCC. Any comparison between published models measured this way is confounded by capacity-dependent inflation, since the artefact is larger for models with more capacity to exploit it.

What Phase 2 could not determine. Which *kind* of SZZ error does the damage. The label-source gap shows that label quality matters; it does not show whether the cost comes from the false positives SZZ invents or the defects it misses. The gap’s failure to track recall (−0.18) rules out the simplest explanation and leaves the question open.

That question is the subject of Chapter 6, and answering it requires an intervention rather than an observation.

Chapter 6 — Phase 3: Diagnosing Learner Behaviour Under Noise

Chapter 5 established that label quality matters to an online learner, and left open which half of the noise is responsible. SZZ makes two kinds of error — it invents defects that do not exist, and it misses defects that do — and Chapter 4 showed the variants trade one for the other. Nothing so far indicates which trade is the right one.

This chapter answers **RQ3**: through what mechanism does label noise degrade an online learner, and does the learner’s imbalance-handling machinery amplify or absorb each error type?

The chapter uses two complementary designs. A **dose-response** experiment injects calibrated noise into clean labels at controlled doses, giving a clean manipulation on a synthetic gradient. A **repair** experiment does the opposite: it takes real B-SZZ labels and uses the reference to surgically correct one error type at a time, giving a controlled intervention on real noise. The second is the stronger design, and it produces the chapter’s headline.

A result to state at the outset, because it shapes how the chapter reads. The hypothesis this project carried into Phase 3 — recorded in its own planning documents — was *false-negative starvation*: that what hurts an online learner is the true defects a conservative labeller withholds. **The controlled experiment refuted it.** Phase 3 was designed as a genuine test rather than a confirmation, and it returned the opposite of the expected answer.

6.1 Method

6.1.1 The repair experiment

Take a project’s real B-SZZ labels. Using the reference labels, construct three counterfactual label vectors:

Condition	Construction
BSZZ	The real B-SZZ labels, unmodified
BSZZ_fp_repaired	B-SZZ with every false positive set to clean (6,565 labels corrected)
BSZZ_fn_repaired	B-SZZ with every false negative set to defective (837 labels corrected)
oracle	The reference labels themselves

Train ORB on each under identical conditions — same stream, same seed, same latency model — and score all four against the reference. The difference between conditions is attributable to the repair, because nothing else differs.

This is an intervention, not an observation. Chapter 5’s label-source comparison contrasted whole labelling procedures that differ in many ways at once. Here a single error type is removed while everything else is held fixed. That is what licenses a mechanistic reading.

It is also oracle-assisted and therefore not a method. Identifying which labels are wrong requires the reference. The repair experiment measures *what would be recovered if one knew*; Chapter 7 asks whether a learner can approximate it without knowing.

Restored false negatives need arrival times. A commit B-SZZ labelled clean has no fix linkage, so a restored positive has no natural delivery date. Arrival times are imputed by sampling the empirical latency distribution, and §6.2.3 tests whether that imputation drives the result.

6.1.2 The dose-response experiment

Starting from clean reference labels, inject noise at doses of 5% to 30% under four profiles, each calibrated from Chapter 4’s measured flip rates:

Profile	Calibration	Character
symmetric	Equal flip rates both directions	Control — the standard assumption
fp_heavy	B-SZZ (ρ_0 0.263, ρ_1 0.359)	False-positive-heavy
mid	RA-SZZ (ρ_0 0.181, ρ_1 0.562)	Intermediate
fn_heavy	L-SZZ (ρ_0 0.067, ρ_1 0.733)	False-negative-heavy

Dose is defined as the expected fraction of *all* commits flipped, with the two class-conditional rates scaled to meet it. §6.2.4 shows that this definition has a consequence severe enough to require its own analysis.

6.1.3 Latency arms

Each dose-response condition is run under three latency regimes:

- **none** — every label delivered immediately after the commit is scored. Plain test-then-train, the true no-latency control.
- **uniform** — every label delayed by exactly $W = 90$ days. A fixed-delay control that removes schedule variation but **not delay**.
- **real** — reconstructed arrival times, with late defect labels delivered first as wrong clean labels and corrected later.

The distinction between **none** and **uniform** is load-bearing and was not present in an earlier version of this work. §6.2.5 records the correction.

6.1.4 Instrumentation

ORB’s internals are traced at every learning step: the Poisson rate λ applied to the arrival, the boost factor, the running prediction bias, and the decayed observed class rate. Alongside these, the label composition of each injected stream is recorded — how many positives it carries and how many of them are genuine. This instrumentation is what turns §6.2.6 from an interpretation into a measurement.

6.1.5 Scale and statistics

21 projects $\times$ 10 seeds $\times$ 4 profiles $\times$ 6 doses $\times$ latency arms = 10,080 dose-response records, plus the repair conditions. Tests are paired at project level ($n = 21$) with Hodges–Lehmann estimates, bootstrap intervals, matched-pairs rank-biserial and Holm correction, matching Chapter 5. Both prequential estimators are recorded and `mcc_avg` is primary.

6.2 Results

6.2.1 The repair verdict

Repair	HL	95% CI	Rank-biserial	Projects	Holm (global)
Remove B-SZZ’s 6,565 false positives	+0.0402	[+0.0217, +0.0520]	+0.844	17/21	0.0019
Restore B-SZZ’s 837 false negatives	−0.0012	[−0.0089, +0.0080]	−0.082	8/21	1.000
The two head to head	+0.0413	[+0.0180, +0.0575]	+0.732	17/21	0.0151

Removing false positives recovers performance. Restoring false negatives recovers nothing — and the interval on that null is tight, so it is an informative null rather than an underpowered one.

A striking detail: FP-repaired B-SZZ reaches a mean of 0.0962 against the reference labels’ own 0.0835. **A precision-repaired heuristic outscores the reference itself** (+0.0127, 14/21, $p = 0.089$). For an online learner under verification latency, label precision is worth more than label completeness. This is an oracle-assisted upper bound, not an achievable labeller, and is framed as such.

6.2.2 The error-matched control changes what the result means

B-SZZ carries 6,565 false positives against 837 false negatives — an 8:1 imbalance. “Repair all of each” is therefore not a fair comparison, and a caveat is not a control. Correcting the **same label mass** on both sides:

Contrast	HL	95% CI	Projects	Holm
FP-repair (matched, ~40/project) vs B-SZZ	+0.0024	[−0.0045, +0.0089]	11/21	0.864
FP-repair (matched) vs FN-repair (all)	+0.0050	[−0.0093, +0.0176]	14/21	0.864

Removing 837 false positives does nothing. Removing 6,565 recovers +0.040. At equal corrected mass the two error types are statistically indistinguishable.

The marginal-value curve reconciles the two results. Repairing false positives in increments of 25/50/75/100% gives a roughly linear return of **+0.147 MCC per 1,000 labels corrected**, so the ~40 per project that false-negative repair could ever match predicts +0.006 — within noise of the +0.002 measured. Nothing is inconsistent; the effect is simply proportional to volume.

The claim this supports, stated exactly. *SZZ’s false positives are what cost the learner, because there are eight times more of them. Not a false positive is individually more harmful than a false negative.* The second sentence is unsupported by this experiment and does not appear in this thesis.

6.2.3 The delivery ceiling: content, not scheduling

A null for false-negative repair admits two readings. The missing labels may not matter — or they may be unable to arrive in time to matter, given that 53% of defect labels arrive after the decision window. Restoring them under three delivery regimes, each against its own anchor so that the contrast isolates restoration from acceleration:

Delivery regime	HL	95% CI	Projects
Realistic (imputed arrival times)	−0.0012	[−0.0089, +0.0080]	8/21
Immediate (available at once)	+0.0037	[−0.0032, +0.0106]	12/21
At the window boundary ($t + 90d$)	−0.0016	[−0.0073, +0.0043]	6/21

Three regimes, three nulls. **Even delivered instantly, the missing labels recover nothing.** The scheduling explanation is ruled out; this is a claim about the content of those labels.

The null is affirmative, not merely unproven. A two one-sided tests (TOST) equivalence procedure — which asks whether an effect is small enough to be declared equivalent rather than merely failing to prove it non-zero — with a margin declared in advance at **±0.02 MCC** (half the measured FP-repair effect) returns **p = 0.0008**. False-negative restoration is *equivalent to no repair*.

6.2.4 Dose-response, and a parameterisation trap

Profile	No latency	Fixed 90-day delay	Realistic delay
fn_heavy	−0.0553	−0.0370	−0.0363
mid	−0.0384	−0.0225	−0.0247
fp_heavy	−0.0313	−0.0196	−0.0205
symmetric	−0.0286	−0.0177	−0.0222

(MCC lost per 10% of labels flipped, fitted on non-degenerate cells only.)

At matched dose, false-negative-heavy noise degrades the learner fastest — which appears to contradict §6.2.1 and does not.

Dose is a fraction of all commits. At equal dose the FN-heavy profile strips far more of the 8.5% minority class. Measured retention of genuine defect labels:

Profile	0.05	0.10	0.15	0.20	0.25	0.30
fn_heavy	0.720	0.440	0.183	0.056	0.010	0.000
fp_heavy	0.938	0.868	0.804	0.739	0.669	0.601

At the highest dose the FN-heavy stream is not empty — it is poisoned. It still carries roughly 204 positive labels, *every one of them false*, because the L-SZZ profile also flips clean commits upward and the majority class is large enough to supply plenty. Fitting a straight line through that region understates the slope and invites a comparison the design cannot support, so slopes are fitted only on cells retaining genuine defect labels, with the survival table published alongside.

The two findings are reconciled by the denominator, and the reconciliation is the substantive point of this chapter. Per *label flipped*, false-negative noise is more damaging, because defect labels are scarce. Per *error actually present in real SZZ output*, false-positive noise dominates, because B-SZZ produces eight times more of them.

6.2.5 Latency masks label quality — a correction

An earlier version of this work concluded that latency does **not** compress sensitivity to label quality. **That conclusion was wrong, and the error was in the design rather than the arithmetic.** The comparison ran between the `uniform` and `real` arms — *both heavily delayed*, differing only in arrival schedule. It tested schedule sensitivity and could not test masking at all.

Against a true no-latency control the picture reverses. Slopes are approximately **50% steeper** without latency, and the separation between the FN-heavy and FP-heavy profiles widens from -0.0158 (real) and -0.0174 (uniform) to **-0.0240** (none) — roughly 40% better separated. Mean MCC at the lowest dose is 0.0995 without latency against 0.0525 with it.

Verification latency both lowers the ceiling and flattens the response to label quality. Under deployment conditions, better labels buy less than they would in a batch setting — which is an independent reason to expect batch-measured noise studies to overstate what label cleaning can deliver.

6.2.6 The mechanism: the boost compensates for scarcity, not for correctness

ORB handles class imbalance by oversampling: each arriving example is presented to each ensemble member $k \sim \text{Poisson}(\lambda)$ times, with λ for the minority class set from the running observed class rate as $\lambda_1 = (1 - r_1)/r_1$, and further amplified when the model's recent predictions are biased against the minority class.

The instrumentation makes the consequence measurable. The mean λ applied to defect-labelled arrivals is an almost perfect inverse function of how many such labels the stream delivers:

- **Spearman $\rho = -1.000$** across profiles at matched dose
- $\rho = -0.70$ to -0.87 within each profile

Profile	Defect labels delivered (mean/project)	Mean λ
fn_heavy @ dose 0.05	113	34.5
symmetric @ dose 0.05	164	23.7
fn_heavy @ dose 0.30	204 (<i>all false</i>)	19.9
symmetric @ dose 0.30	436	8.3

ORB's boost compensates for defect-label scarcity, not for false negatives — it cannot tell the difference. It amplifies whatever positive labels arrive, and amplifies them hardest exactly when they are rarest.

This is the mechanism the chapter was looking for, and it explains both halves of the result:

Why restoring false negatives recovers nothing. The learner has already compensated for their absence. Removing defect labels raises λ on those that remain, which partially replaces the lost gradient mass. Adding the missing labels back supplies something the mechanism had already approximated.

Why removing false positives recovers a great deal. A false positive receives the same amplification as a true one — the boost has no facility for discounting a label it does not trust. **The machinery that corrects class imbalance is the machinery that magnifies wrong labels**, and it magnifies them hardest precisely when defect labels are scarcest, which under verification latency is always.

6.3 Discussion — answering RQ3

Through what mechanism does label noise degrade an online learner? Through amplification, not through starvation. ORB's oversampling rate is set by the observed rate of defect labels, so it responds to the *quantity* of positive labels and is blind to their *quality*. False positives are therefore not merely learned — they are learned harder than a balanced learner would learn them, and hardest under exactly the conditions that make deployment realistic.

Does the learner amplify or absorb each error type? It absorbs false negatives, by construction: their absence raises λ , which partially compensates. It amplifies false positives, also by construction, and has no mechanism to do otherwise.

Which error dominates in practice? False positives — **because of their volume, not their unit severity**. This is the distinction the error-matched control enforces, and it is the difference between a claim this thesis can defend and one it cannot.

6.3.1 Design requirements extracted for Phase 4

1. **Target false positives, not false negatives.** Restoring missing defect labels is equivalent to no repair. This retires the rescue mechanism Phase 4 was originally designed around.
2. **Operate at volume.** The benefit is roughly linear at +0.147 MCC per 1,000 corrected labels, so an intervention touching a few dozen labels per project will do nothing detectable.
3. **Expect the benefit to track each label source's false-positive count.** This is a falsifiable ordering prediction across six variants whose counts are known in advance, and it becomes Phase 4's sharpest test.
4. **Anticipate interference with the boost.** Because λ is an inverse function of delivered defect-label supply, any filter that suppresses positives raises λ for the survivors. Whether that helps or hurts is not obvious and must be registered as a risk rather than assumed.
5. **Guard the clean-label case.** Nothing here shows a filter can tell a false positive from a correct label the model has not yet learned. Non-degradation on clean labels is the acceptance bar, not a secondary check.

6.3.2 What this chapter licenses, and what it does not

Licensed. That the volume of SZZ's false positives is what costs an online learner performance on this corpus; that restoring its false negatives is equivalent to no repair under every delivery regime tested; that verification latency compresses sensitivity to label quality; that ORB's oversampling responds to label quantity and is blind to label quality.

Not licensed. That a false positive is individually more harmful than a false negative — the error-matched control rules this out. That these conclusions transfer to learners without an imbalance-driven oversampling mechanism; the mechanism identified is specific to that architecture. That a deployable method can capture the measured +0.040, since every repair here is oracle-assisted.

The last of these is Chapter 7's question, and its answer is not the one the design requirements above would predict.

Chapter 7 — Phase 4: Noise-Aware ORB

Chapter 6 established what an online learner cannot survive: the volume of false positives SZZ invents, amplified by the very mechanism that corrects class imbalance. It also showed what recovering from that is worth — **+0.040 MCC** — using the reference labels to remove exactly the wrong ones.

This chapter answers **RQ4**: can a learner approximate that repair *without* the reference, and without sacrificing performance when the labels are clean?

The answer is no, and this chapter reports that at full strength. The intervention is adoptable — it does not damage clean labels — but no confirmatory test survives multiplicity correction. This is a **pre-registered characterised negative**, and the chapter is organised to explain not merely that it failed but *why*, because the reason is more informative than a success would have been.

7.1 Pre-registration, and a registration that was rewritten

7.1.1 Why Phase 4 is pre-registered and Phases 1–3 are not

Phases 1–3 are exploratory. Their hypotheses were formed during analysis, their test families were defined after seeing the data, and every result is labelled accordingly, with a conservative global multiplicity correction applied so that no claim depends on how the families were drawn.

Phase 4 is different in kind. It proposes a method and asks whether it works — a question that invites the analytic flexibility pre-registration exists to constrain. Its hypotheses, its primary model, its acceptance bar and its multiplicity correction were therefore committed to version control **before the first Phase 4 run**, and the commit history timestamps that claim independently of this chapter's narration.

7.1.2 The registration was revised once, and the revision is part of the record

The original registration made a **false-negative rescue** mechanism the headline: when the ensemble confidently disagrees with a clean label, train the commit as a provisional positive. That design followed directly from the starvation hypothesis Phase 4 was planned under.

Chapter 6 refuted that hypothesis. Restoring genuinely missing defect labels is *equivalent to no repair* (TOST $p = 0.0008$). A mechanism that manufactures positives therefore targets an error shown to cost nothing — and worse, every incorrect rescue manufactures exactly the error type Chapter 6 showed to be costly.

Because Phase 4 had produced **zero results** at that point, the registration was rewritten and re-committed. **That is the only moment at which revising a pre-registration is legitimate**: before any outcome is visible. Revising afterwards is the practice pre-registration exists to prevent, and the distinction between the two is entirely a matter of timing, which is why the timing is recorded rather than asserted.

The rescue mechanism was not discarded. It was **demoted to a mechanism probe carrying a predicted sign**, so that it would test Chapter 6’s account rather than merely be excluded by it.

7.1.3 The registered tests

Tag	Test	Prediction
ND	Filter vs baseline on reference labels	No significant drop (TOST, ± 0.02). The acceptance bar
H1	Filter beats baseline on B-SZZ	Positive (6,565 false positives — the most available)
H2a/b	Filter beats baseline on MA-SZZ and AG-SZZ	Positive (5,030 and 4,786)
H3	Benefit rank-correlates with each variant’s false-positive count	$\rho > 0$, one-sided
MP	Rescue vs baseline on every SZZ source	≤ 0 on all six
R1	λ self-antagonism arm	suppress underperforms observed

H3 is the sharpest test in the chapter, and it exists because of Chapter 6’s error-matched control. If the benefit is driven by false-positive *volume*, it should track how many false positives each source contains — a predicted ordering over six variants whose counts were fixed in advance (6565, 5030, 4786, 4531, 2316, 1665). An ordering is far harder to satisfy by chance than a binary comparison.

ND is the acceptance bar, not H1. A filter that wins on noisy labels and damages clean ones is not deployable, because a practitioner cannot know in advance which case they are in.

7.2 Design

7.2.1 The intervention

`FPFilterORB` extends ORB with train-time false-positive suppression. A defect-labelled arrival that the ensemble confidently contradicts is treated as a suspected SZZ false positive: trained at weight ϵ (with $\epsilon = 0$ discarding it) and excluded from the boost. **Clean-labelled arrivals are untouched**, since Chapter 6 showed that intervening on the negative side recovers nothing.

7.2.2 Two decision rules, and why the obvious one fails

The natural rule — and the one originally specified — is a running quantile: suppress a positive whose probability falls in the lowest q of recently delivered positives. It requires no calibration, which is its appeal.

Measured on this corpus that rule is degenerate. The ensemble’s members agree almost completely, so the predicted probability is bimodal at 0 and 1, and the 10th percentile of a trailing window is **0.0000 in nearly every segment of every stream**. A strict comparison against a threshold of zero can never fire: on `opennlp/B-SZZ` it suppressed **0 of 75** positive arrivals. The rule was not wrong in conception; it was wrong in its comparison operator, and a non-strict comparison repairs it.

The alternative is a fixed threshold τ . §7.4.1 shows it fails for a deeper reason.

```

FPFilterORB.learn_one(x, y):
    if y == 0:
        ORB.learn_one(x, y); return          # negatives untouched

    p1 ← ensemble_probability(x)
    thr ← τ                                if mode = fixed
        quantile(window, q) if mode = quantile and |window| ≥ min_pos
        1                    otherwise      # inert during warm-up
    suspect ← (thr ≠ 1) and (p1 < thr if fixed else p1 ≤ thr)
    window.append(p1)

    if suspect:
        if rate_update = "observed":
            r1 ← decay·r1 + (1-decay)·1      # distrust the LABEL, not the class rate
        if ε > 0:
            k ~ Poisson(ε); update_members(x, 1, k)
        return                                # no boost, no full-weight update

    ORB.learn_one(x, y)

```

Complexity. One additional ensemble forward pass per defect-labelled arrival and one quantile over a bounded window, so $O(M + \log W)$ per positive with M ensemble members and window W , against ORB's $O(M \cdot \lambda)$ update. Since positives are 8.5% of arrivals and λ routinely exceeds 20, the filter's cost is negligible.

Warm-up is counted in positive labels, not arrivals. Positives are what is scarce: under realistic latency some projects deliver fewer than 30 across an entire stream, and a quantile over an empty window is undefined. Below `min_pos` the filter is inert and the model is plain ORB — the safe direction.

7.2.3 The registered risk: λ self-antagonism

Chapter 6 measured ORB's oversampling rate to be a near-perfect inverse of delivered defect-label supply ($\rho = -1.000$), and the mechanism is explicit in the update: $\lambda_1 = (1 - r_1)/r_1$. **Every positive this filter suppresses therefore raises λ for the positives that survive it.** The filter works against the machinery it is bolted onto.

The `rate_update` switch controls whether it does:

- **observed** — r_1 is updated as though the label had been accepted. Suppressing a label expresses distrust of *that label* without telling the ensemble the defect class is rarer than it is. λ is left alone.
- **suppress** — the arrival is skipped entirely, r_1 drifts down, λ rises.

Registered prediction: suppress underperforms observed, with the gap widening as filter rate rises. A second risk — that a suppressed positive is never learned, so its probability stays low and it is suppressed again — is instrumented rather than assumed: every decision is traced.

7.3 Experimental setup

Models. OOB and ORB as baselines; `NA(fp_filter)` as the registered primary; `NA(fp_filter/suppress)` as the R1 arm, carrying identical hyperparameters with only the switch flipped; `NA(damp)`, the prior confidence-damping defence, for comparison; `NA(rescue)` as the MP probe.

A composed `NA(fp_filter+damp)` arm appeared in the first draft of the registration but was **never implemented** — the placeholder returned a duplicate of the primary, which would have reported one model under two names and made the ablation look richer than it is. It was dropped before any Phase 4 run, at the same moment and for the same reason

the registration itself could legitimately be revised.

Conditions. Reference labels (the ND gate), all six SZZ variants, and a 20% injected-noise condition bridging to Chapter 6.

Held-out projects. `commons-scxml`, `opennlp` and `commons-math` are excluded from every reported number. Hyperparameters were chosen by looking at them, so including them would leak the tuning set into the results. Reporting is therefore on **18 projects**.

Statistics. Project-paired Wilcoxon on `mcc_avg`, Hodges–Lehmann with bootstrap interval, matched-pairs rank-biserial, Holm within the registered family. Both estimators recorded.

7.4 Results

7.4.1 Tuning, and what the acceptance bar caught

Forty-two configurations were evaluated on the held-out projects under a selection rule declared before the sweep: apply the ND gate **first**, then maximise gain on B-SZZ, then prefer the more conservative filter. `rate_update` was deliberately **not** tuned — it is the R1 arm, and optimising it would convert a registered prediction into a fit.

Twenty-four configurations passed the gate, and every one of them uses the quantile rule. Every fixed-threshold configuration failed, several catastrophically: on the reference arm, where every positive label is correct and there is nothing to remove, $\tau = 0.35$ suppressed 57% of defect labels and cost **-0.0989** MCC.

The reason generalises well beyond this model. A fixed confidence threshold suppresses however many positives the model happens to be unsure about, with no reference to how many false positives the source actually contains. It cannot distinguish “*this label is probably wrong*” from “*my model has not learned this pattern yet.*” A quantile rule removes about q by construction, so its worst case is bounded. **An intervention on scarce labels must be bounded in size, not in confidence.**

The frozen configuration is quantile with $q = 0.30$, $\epsilon = 0.1$, `min_pos` = 30, `rate_update` = `observed`. Extending the sweep to $q = 0.40$ and 0.50 lowered the B-SZZ gain to 0.0402 and 0.0288, confirming $q = 0.30$ as an interior optimum rather than a grid edge.

7.4.2 The registered verdict

Eighteen reporting projects.

Test	HL	95% CI	Projects	<i>p</i>	Holm
ND — non-degradation on reference labels	−0.0001	[−0.0091, +0.0081]	5/18	0.807	PASS
H1 — filter beats baseline on B-SZZ	+0.0166	[+0.0009, +0.0328]	12/18	0.054	0.215 ✗
H2a — on MA-SZZ	+0.0110	[−0.0005, +0.0235]	13/18	0.074	0.221 ✗
H2b — on AG-SZZ	+0.0072	[−0.0075, +0.0221]	12/18	0.246	0.492 ✗
H3 — benefit tracks false-positive count	$\rho = 0.714$	—	6 variants	0.055	✗

The gate passes; nothing else does. Every confirmatory test points in the predicted direction and not one survives correction.

The honest reading is narrow. The filter is **adoptable** — it does not damage clean labels, which is the property that gates deployment — but it is **not demonstrated to work**. Four independent tests agreeing in direction is worth a sentence; it is not a result, and this thesis does not treat it as one. H1’s interval barely excludes zero and its raw *p* of 0.054 would not have survived even without correction.

The hold-out protocol paid for itself. On the three tuning projects the filter gained **+0.045** on B-SZZ. On the 18 reporting projects it gains **+0.017** — an optimism factor of **2.7×**. Had the hyperparameters been selected on the reporting set, this chapter would have claimed an effect nearly three times its real size, and would have been wrong.

7.4.3 The mechanism probe was refuted, 0 for 6

Rescue was registered with an explicit prediction that it would be **at or below zero** on every SZZ source. It is **positive on all six**:

Source	Rescue – baseline	95% CI	Projects	<i>p</i>
L-SZZ	+0.0166	[+0.0057, +0.0289]	13/18	0.006
MA-SZZ	+0.0142	[+0.0013, +0.0266]	12/18	0.027
AG-SZZ	+0.0117	[−0.0007, +0.0270]	13/18	0.090
B-SZZ	+0.0116	[+0.0033, +0.0262]	13/18	0.024
R-SZZ	+0.0068	[−0.0033, +0.0202]	12/18	0.246
RA-SZZ	+0.0002	[−0.0164, +0.0213]	8/18	1.000

The registration stated that if rescue helped, the mechanism account would need revision. **It helps, and the revision is owed.**

The candidate reconciliation, and why it is not yet a claim. Chapter 6 restored the *reference-known* missing positives; rescue adds *model-confident* ones. These are different sets — rescue is closer to self-training on high-confidence commits than to label correction. So “restoring what SZZ missed recovers nothing” and “adding confidently-predicted defects helps” are not contradictory, **provided** the benefit comes from label *supply* rather than label *accuracy*.

That explanation was tested directly. The Spearman correlation between a condition's delivered positive supply and the filter-minus-rescue advantage is $\rho = +0.357$, $p = 0.385$. **The supply account is not supported**, and it is recorded here as an open question rather than presented as an explanation.

7.4.4 R1: a clean null in both directions

Condition	observed – suppress	95% CI	p
B-SZZ	-0.0016	[-0.0075, +0.0045]	0.61
Reference	+0.0002	[-0.0018, +0.0031]	0.51

The registered prediction is not supported, and neither is its converse. λ *does* rise when positives are suppressed — the coupling is real and was measured at $\rho = -1.000$ — but at this filter rate it has no detectable effect on performance. The trace confirms the feedback loop exists: the two settings produce *different filter rates on identical inputs*. It simply does not matter at this magnitude.

7.4.5 An uncomfortable finding, reported rather than smoothed

The prior confidence-damping arm — the one the re-registration demoted — **outperforms the new primary on six of eight conditions**, including B-SZZ (0.0874 against the baseline's 0.0551).

Because damping was not the registered primary, every such contrast is exploratory. Corrected across the 21 exploratory contrasts, only damping-beats-baseline on B-SZZ survives global Holm: +0.0329, CI [+0.0194, +0.0456], 15/18, $p = 0.0040$. **The defensible statement is that damping leads numerically and is established on one label source.**

The re-registration was correctly motivated by Chapter 6 and correctly timed before any run, and it still bet on the wrong arm. The commit history records this either way, so the thesis states it.

7.5 Discussion — answering RQ4

Can a learner recover performance lost to SZZ noise without reference labels? Not demonstrably, on this corpus, with this intervention.

	Recovers
Chapter 6 FP-repair, using the reference to remove exactly the wrong labels	+0.040
Chapter 7 filter, approximating it without the reference	+0.017 (not significant)

The gap between those numbers is the price of not knowing which labels are wrong, and it is the most informative quantity in this chapter. Two things make it large.

A confidence signal cannot separate the two cases that matter. At its tuned setting the filter suppresses 30% of defect labels on B-SZZ — and still **15% on reference labels, where there is nothing to remove**. Low predicted probability means “this label disagrees with what I have learned”, which is equally consistent with a wrong label and with a correct label describing a pattern not yet learned. Under an 8.5% positive rate with a model still learning, the second case is common.

Defect labels are too scarce for the intervention to be cheap. Under realistic latency a project delivers a mean of 75 defect labels across an entire stream, with ORB's λ climbing to 62 to compensate. Every suppressed label is drawn from that budget. Chapter 6's repair could afford to remove 6,565 labels because it removed only wrong ones; a heuristic filter pays for its precision in correct labels discarded.

When it helps and when it does not. The direction is consistent — positive on all four registered tests and ordered roughly by false-positive count — so the mechanism identified in Chapter 6 is not contradicted. The effect is simply smaller than the design can resolve at $n = 18$. On the most false-positive-heavy source the point estimate is +0.017 against a baseline near 0.055, which would be practically meaningful if it were established; on the cleanest sources it is indistinguishable from zero, exactly as the volume account predicts.

What the negative result is worth. More, in this case, than a positive one. A working filter would have shown that this particular heuristic recovers some of this particular loss. The negative shows something more general: **the information required to build the defence is not available to a learner at training time.** Knowing *which kind* of error hurts — Chapter 6's contribution — does not confer the ability to identify *which instances* are that error. The repair experiment's +0.040 is an upper bound requiring knowledge no deployed system has.

The practical implication is the thesis's, not a hedge: **on this evidence, improving the labels is worth more than improving the learner.**

7.5.1 What this chapter licenses, and what it does not

Licensed. That train-time false-positive filtering is adoptable but not demonstrated effective on this corpus; that a fixed confidence threshold is actively harmful on clean labels and the failure mode is understood; that the rescue mechanism helps, contradicting a prediction derived from Chapter 6; that held-out tuning was necessary, with a measured optimism factor of 2.7 $\times$.

Not licensed. That confidence-based filtering cannot work in principle — one design was tested at one operating point. That damping is superior to filtering; it leads numerically but is established on one source and was not pre-registered. Any explanation of why rescue helps: the supply account was tested and failed, and no replacement has been tested.

Chapter 8 — Discussion

The four phases were designed to answer four questions in sequence. In the event, two of them returned answers the design did not anticipate, and one returned a negative. This chapter reads the four results as a single argument, states what follows for researchers and for practitioners, and gives the full treatment of validity threats.

8.1 Synthesis across the research questions

8.1.1 The four answers

RQ1 — How much do SZZ variants disagree, and in which direction? Substantially, and structurally. No variant exceeds 27.2% **precision** against the reference. Variants agree with one another at κ up to **0.933** while agreeing with the reference at κ **0.158–0.202** — four to six times more strongly with each other than with the thing they are estimating. Bias is bifurcated: B-SZZ over-flags ($p_0 = 0.263$), L-SZZ and R-SZZ under-flag ($p_1 = 0.733, 0.700$). **Two decades of refinement have relocated error rather than reduced it**, moving it from the false-positive column to the false-negative column.

RQ2 — How does label source shift measured performance across regimes? It shifts it significantly — 0.034 to 0.064 MCC under realistic streaming, against a deployable baseline near 0.097. **But the evaluation protocol shifts it far more.** Random k -fold inflates a random forest by **+0.137 MCC**; scoring against the training labels inflates it by a further **+0.230**. The two protocol effects together are roughly nine times the label-source effect.

RQ3 — Through what mechanism does noise degrade an online learner? Through amplification rather than starvation. Repairing B-SZZ's false positives recovers **+0.040 MCC**; restoring its false negatives is *equivalent to no repair* (TOST $p = 0.0008$) under realistic, immediate and at-window delivery alike. The mechanism is measurable: ORB's oversampling rate is a near-perfect inverse of delivered defect-label supply ($\rho = -1.000$). The boost compensates for label *scarcity* and cannot distinguish a scarce-but-correct stream from a scarce-and-wrong one.

RQ4 — Can a learner recover that loss without the reference? Not demonstrably. The pre-registered non-degradation gate passes, so the intervention is adoptable; every confirmatory test points the predicted way and **none survives correction**. The oracle-assisted repair recovers +0.040; the deployable approximation recovers +0.017 and is not significant.

8.1.2 The noise pipeline, read end to end

The four results compose into one account of how a number becomes untrustworthy.

Stage 1 — The labels are wrong, asymmetrically. Tangled commits cause SZZ to blame lines that fix nothing. Filters remove some of that over-collection and, for the aggressive variants, more than half of what they remove is correct.

Stage 2 — The evaluation hides it. Because the labels are heuristic, models are scored against the same heuristic that trained them. A high-capacity model that fits the heuristic's error structure is rewarded for doing so. **This is only possible because the labels are wrong:** with ground truth there would be nothing to score against but truth, and the +0.230 gap could not exist. The two problems are not independent — the labelling failure creates the evaluation failure.

Stage 3 — Deployment removes the protections. Under streaming with verification latency, the learner's imbalance machinery amplifies whatever defect labels arrive. Most of the surviving labels are false positives, and they are amplified hardest exactly when defect labels are scarcest — which, given that 53% arrive after the decision window, is always.

Stage 4 — Knowing this is not sufficient to fix it. Identifying *which kind* of error hurts does not confer the ability to identify *which instances* are that error. A confidence signal cannot separate “this label is wrong” from “I have not learned this pattern yet”, and under an 8.5% positive rate the second case is common.

The unifying quantity is deliverable precision. What an online learner needs is not correct labels in the abstract but correct labels that *arrive* — and among those that arrive, false positives are the error it cannot defend against, because its own imbalance mechanism amplifies them. Every result in this thesis is a corollary of that sentence, including the retirement of the starvation hypothesis it began with.

8.1.3 Statistical significance and practical significance

The measurement critique is statistically overwhelming: rank-biserial +1.000 on the leakage effect (21 of 21 projects), +0.991 on the self-scoring gap, intervals excluding zero by wide margins, survival under global correction.

Practical significance is more modest and should be stated as such. Even with perfect labels and an honest protocol, deployable MCC on this corpus is approximately **0.097**. Repairing every false positive raises it to 0.096 from 0.058 — a 66% relative gain on a small absolute base. **This thesis does not claim that better labels would make JIT-SDP good.** It claims they would make it measurably less self-sabotaging, and that the field's reported numbers overstate what it currently achieves by roughly a factor of four.

The low ceiling is itself a finding. A field reporting 0.41 and delivering 0.10 has a calibration problem independent of whether 0.10 is useful.

8.2 Implications

8.2.1 For researchers building datasets

Stop treating inter-tool agreement as validation. Variants agreeing at κ 0.93 while agreeing with the reference at κ 0.16 is the clearest result in Chapter 4. Agreement between procedures that share a blame step and a set of assumptions measures shared failure modes.

Report the denominator. A variant scored on the candidates it emits will appear more precise than one scored on the whole corpus. Chapter 4’s comparison is only interpretable because every variant was scored over an identical 27,319-commit population.

Report both flip rates, not a single noise rate. ρ_0 and ρ_1 differ by a factor of eleven across the variants studied here. Any downstream noise model calibrated on a single scalar will be wrong for five of the six.

Distinguish rate from volume. At an 8.5% positive rate, a ρ_0 of 0.263 produces 6,565 wrong labels while a ρ_1 of 0.733 produces 1,710. Chapter 6’s central finding turns entirely on this distinction, and the error-matched control that established it is cheap to run.

8.2.2 For researchers reporting results

Random k -fold on temporally ordered data is not defensible, and the inflation it produces is *larger for models with more capacity to exploit it*. Comparisons between model families measured this way are confounded, and the confound flatters exactly the models the field most wants to promote.

Score against something other than the labels you trained on, or state plainly that the number measures fidelity to a heuristic. Where no independent reference exists, the honest description of a self-scored result is “reproduces SZZ at MCC x ”, not “detects defects at MCC x ”.

Name the prequential estimator. A terminal fading value and a trajectory mean disagree on the sign of one comparison in this thesis and would have supported a claim that had to be withdrawn. Neither is canonical for MCC, which is not a decomposable loss.

Report multiplicity correction globally when families were not pre-declared. The distinction costs one column and removes an entire class of objection.

8.2.3 For practitioners

Expect roughly 0.10 MCC, not 0.40. If a JIT-SDP tool is evaluated on your own repository under an honest protocol and returns something near 0.10, it is performing as well as the state of the practice on this corpus — not failing.

Label quality is worth more than model sophistication. Reference labels beat every SZZ variant under streaming; a single-feature logistic regression beats a 100-tree forest under a chronological split. Investment in knowing which commits actually introduced defects dominates investment in the learner.

If you must use SZZ, prefer the permissive variant and accept the false positives — B-SZZ carries the smallest penalty of the six under streaming, and the aggressive filters discard more correct answers than incorrect ones. This runs against the direction of variant development and is stated because the data support it.

Do not deploy a confidence-based label filter on the strength of this work. Chapter 7 tested one, it passed the non-degradation gate, and it did not demonstrably help. A fixed-threshold variant was actively harmful on clean labels.

8.3 Threats to validity

8.3.1 Construct validity

The reference is not ground truth, and every comparison inherits that. It is `git blame` seeded with human-verified fix lines. Comparisons against it isolate the cost of tangled commits, not total labelling error. **All measured noise is therefore a lower bound:** blame error is present on both sides and cancels.

The reference condition's label timing is SZZ-derived. This is the most serious threat in the study. The published labels carry no native fix-to-inducing linkage, so arrival times were reconstructed from the union of SZZ mappings. The reference is blame-derived in construction *and* SZZ-derived in timing. A coverage sensitivity analysis (67.8% → 100%) bounds part of the exposure; the timing itself is not bounded, and every streaming result in Chapters 5–7 inherits it.

The corpus is SZZ-shaped. Changes adding no new lines were excluded by the dataset authors, so defects introduced purely by deletion cannot appear. Separately, because the reference is blame output, every reference positive is blame-reachable by construction, so blame-unreachability is not an available explanation for any measured false negative. This limit was declared before Phase 3 ran.

8.3.2 Internal validity

Phase 3's repair conditions are not error-matched, and the matched control changed the claim. Repairing “all of each” compares 6,565 corrections against 837. At equal mass the two error types are indistinguishable. The supported claim is about error *volume*; the per-label claim is explicitly disclaimed.

Phase 3's dose parameterisation is not class-balanced. Dose is a fraction of all commits, so the FN-heavy profile strips far more of the minority class at equal dose, and at the highest dose leaves a stream whose defect labels are *all false*. Slopes are fitted only on cells retaining genuine defect labels.

One published conclusion was wrong and is corrected rather than removed. An earlier version reported that latency does not compress sensitivity to label quality. That comparison ran between two uniformly delayed arms and could not test the question; against a true no-latency control the conclusion reverses.

The latency decomposition is unresolved, not null. A full 2×2 was built after the original decomposition was found to be unidentified. Every contrast, including the interaction, has an interval spanning zero.

A label-vintage defect was found and fixed mid-study. Models had trained on labels one commit older than the noise rates reported, affecting 1.0–6.5% of commits per variant. The remediation ships as an executable gate that runs before any compute.

Phase 4's held-out projects were not randomly assigned. They were chosen after Phases 1–3 were analysed. Nothing in Phase 4's reported numbers depends on them, but the assignment was purposive.

8.3.3 External validity

One benchmark family. Twenty-one Apache Java repositories of a single dataset lineage. Java-specific idioms, Apache-specific review culture and the dataset's own inclusion criteria are all uncontrolled.

Project heterogeneity is real but does not drive the results. Projects span 544–4,026 commits and 19–335 defects and are weighted equally. Excluding those below a 25-defect floor moves no headline estimate by more than 0.002 MCC and all four Tier 1/Tier 2 claims still survive correction.

The mechanism is architecture-specific. The amplification account applies to learners whose oversampling responds to observed class rate. A learner without that machinery would have a different failure mode, and Chapter 6’s conclusions should not be transferred to one.

8.3.4 Conclusion validity

Effects below roughly 0.04 MCC are not resolvable at $n = 21$. This is load-bearing for the latency decomposition and for Phase 4’s H1 (+0.017), and both are reported as unresolved rather than as null.

The exploratory comparison count is large, which is why global correction is reported alongside within-family correction and why headline claims are those surviving the global column.

Phase 4’s negative could be a power failure rather than an absence. Four registered tests agreeing in direction, with H1’s interval barely excluding zero, is consistent with a real effect too small to resolve at $n = 18$. The chapter says “not demonstrated to work”, not “shown not to work”, and the distinction is deliberate.

8.3.5 Threats that were anticipated versus discovered

Anticipated in Chapter 3 and unchanged: the reference’s construct limits, the single benchmark family, the equal weighting of unequal projects, the resolution floor.

Discovered during the work, and recorded because the pattern is itself informative: the error-matching confound, the dose parameterisation trap, the degenerate quantile rule, the label-vintage cache defect, the unidentified latency decomposition, and the no-latency arm that was not one. Every one of these was found by a control that had been built for a different purpose, or by a reviewer asking what a caveat was doing in place of an experiment. **A caveat is not a control,** and five of the six corrections in this thesis exist because that distinction was eventually enforced.

Chapter 9 — Conclusion and Future Work

9.1 What this thesis set out to do

The thesis began with a question about cost: how much does SZZ label noise cost a just-in-time defect prediction model? The question presumes that the cost is the interesting quantity, and that measuring it would tell the field how much to invest in better labels.

The four phases answered that question — under realistic streaming evaluation, training on SZZ rather than the reference costs between 0.034 and 0.064 MCC against a deployable baseline near 0.097 — and in answering it exposed a larger one. **Most of what the field reports is not capability but measurement artefact,** and the artefact is created by the same labelling weakness the thesis set out to quantify.

9.2 Contributions

A characterisation of SZZ noise as a class-conditional channel. Six variants scored over an identical 27,319-commit denominator, with a precision ceiling of 27.2%, bifurcated bias (p_0 0.067–0.263, p_1 0.359–0.733), and inter-variant agreement up to κ 0.933 against reference agreement of κ 0.158–0.202. **Variants share failure modes; their**

agreement is not evidence of correctness. The flip-rate pairs are released as a calibration artefact for downstream noise modelling.

A quantification of evaluation-protocol inflation, separated from label effects. Random k -fold inflates a high-capacity model by +0.137 MCC (21/21 projects); scoring against the training labels inflates it by a further +0.230. **The protocol effects are roughly nine times the label-source effect**, and the inflation scales with model capacity, so it flatters exactly the models the field most promotes. The inflation ladder — 0.4129 under the field’s standard configuration, 0.0970 under a defensible one — is the thesis’s single most consequential number.

A controlled identification of which SZZ error costs an online learner, with its mechanism. Repairing false positives recovers +0.040 MCC; restoring false negatives is *equivalent to no repair* (TOST $p = 0.0008$) under three delivery regimes. An error-matched control establishes that this is a **volume** effect, not a per-label one. The mechanism is measured rather than inferred: the learner’s oversampling rate is a near-perfect inverse of delivered defect-label supply ($\rho = -1.000$), so **the machinery that corrects class imbalance is the machinery that magnifies wrong labels**.

A pre-registered negative result, with the reason it failed. A train-time false-positive filter is adoptable but not demonstrably effective: the non-degradation gate passes, no confirmatory test survives correction. The gap between the oracle-assisted repair (+0.040) and the deployable approximation (+0.017) is **the price of not knowing which labels are wrong** — a confidence signal cannot separate “this label is wrong” from “I have not learned this yet”, and defect labels are too scarce under latency for that confusion to be cheap.

A reproducibility apparatus. The full pipeline runs on a continuous-integration runner from two small committed files, gated by an assertion that recomputes the training data’s confusion matrix and refuses to spend compute if it disagrees with the reported noise rates. The gate exists because the study encountered exactly the label-provenance failure it was written to study.

9.3 Limitations

The reference is `git blame` seeded with verified fix lines, not ground truth, so all measured noise is a **lower bound** and every comparison isolates tangling rather than total labelling error. The reference condition’s label *timing* is SZZ-derived, which is the study’s most serious construct threat and is not bounded by any analysis performed here. The corpus is one benchmark family of 21 Apache Java repositories and excludes deletion-introduced defects by construction. At $n = 21$, effects below roughly 0.04 MCC are unresolvable, which leaves the latency decomposition unresolved rather than null. The mechanism identified in Phase 3 is specific to learners whose oversampling responds to observed class rate.

9.4 Future work

Validate against a non-blame reference. The lower-bound claim is currently an argument, not a measurement. A developer-informed oracle — one in which the *developer* named the defect-introducing commit in the commit message, with no blame step in the chain — would test it directly. Such datasets exist but contain only fix-to-inducing links and no clean commits, so they cannot train a model; they can, however, validate Phase 1 against a reference that does not share SZZ’s failure mode. **A caution for anyone attempting this:** precision figures reported against such oracles are conditional on fixes already known to have an inducing commit, whereas this thesis scores over an entire project history. The two are not comparable without reconciling the denominator, and the difference is large enough to look like a contradiction.

Resolve the rescue anomaly. Phase 4's mechanism probe was refuted: adding model-confident defect labels helps, while restoring genuinely missing ones does not. The obvious reconciliation — that the benefit comes from label supply rather than accuracy — was tested and failed ($p = +0.357$, $p = 0.385$). This is the sharpest open question the thesis leaves, and it is well posed: two interventions that both increase positive-label supply have opposite effects, and no current account explains why.

Pre-register the damping arm. Confidence damping outperformed the registered primary on six of eight conditions but was not the registered primary, so only one contrast survives correction. It deserves its own registration rather than promotion on exploratory evidence — which is precisely the practice this thesis criticises elsewhere.

Effort-aware evaluation. JIT-SDP is deployed as an inspection-prioritisation aid, so recall at a fixed inspection budget is closer to the practitioner's objective than MCC. Whether the inflation ladder has the same shape under effort-aware metrics is unknown and directly testable from the committed predictions.

Cross-project and cross-language streams. Every conclusion here is within-project and within-language. Whether the amplification mechanism survives a transfer setting, where the positive rate and the feature distribution both shift, is untested.

Latency-aware label acquisition. If false-positive volume is what costs the learner and 53% of defect labels arrive too late to help, the highest-value intervention may be neither a better labeller nor a better learner but a *faster* one — reducing the interval between defect introduction and label availability. Nothing in this thesis tests that, and it follows more directly from the results than any method it did test.

9.5 Closing

The thesis asked how much SZZ label noise costs a defect prediction model. The answer turned out to be less interesting than a question it exposed along the way: **how much of what the field reports is measurement artefact rather than capability?**

On this corpus, most of it. A number that reads as 0.41 under the field's standard protocol is 0.10 once the model is scored against something other than the heuristic that trained it and evaluated in an order that time permits. That gap — roughly four-fifths of the reported signal — is quantified here, bounded by confidence intervals, corrected for multiple comparisons, stable across every analytical choice tested, and reproducible from a single commit hash.

The last two phases asked the obvious follow-up — *so fix it* — and returned something more useful than a working method. The damage is done by the sheer volume of false positives SZZ invents, not by the defects it misses; the missing defects are provably worth nothing to recover. But a learner cannot act on that knowledge, because it cannot tell a wrong label from an unlearned pattern, and verification latency leaves it too few defect labels to risk discarding any.

That is a negative result, pre-registered as such, and it is reported at full strength rather than reframed. It is also the most informative finding after the measurement critique itself, because it locates the difficulty precisely: **the problem is not that nobody has built the right defence, but that the information required to build one is not available to a learner at training time.**

Which returns the field to where the thesis began, with a clearer instruction than it started with. **Improving the labels is worth more than improving the learner.**